

Lecture 11

Dynamic Programming



Slides are based on the textbook and its notes

Overview

- ❑ *Dynamic Programming*, like divide-and-conquer method, solves problems by combining the solution to subproblems. (“Programming” in this context refers to a tabular method, not to writing computer code.)
- ❑ Divide-and-conquer algorithms partition the problem into disjoint subproblems, solve the subproblems recursively, and then combine their solutions to solve the original problem.
- ❑ In contrast, dynamic programming applies when the subproblems overlap – that is, when subproblems share subproblems.
- ❑ In this context, a divide-and-conquer algorithm does more work than necessary, repeatedly solving the common subsubproblems.
- ❑ A dynamic programming algorithm solves each subsubproblem just once and then saves its answer in a table, thereby avoiding the work of recomputing the answer every time it solves each subsubproblem.

Overview

- ❑ Used for optimization problems:
 - Find a solution with the optimal value.
 - Minimization or maximization.
- ❑ To develop a dynamic programming algorithm, follow a sequence of four steps:
 - Characterize the structure of an optimal solution.
 - Recursively define the value of an optimal solution.
 - Compute the value of an optimal solution, typically in a bottom-up fashion.
 - Construct an optimal solution from computed information.

Optimal binary search trees

- ❑ Suppose that you are designing a program to translate text from English to French. For each occurrence of each English word in the text, you need to look up its French equivalent.
- ❑ You can perform these lookup operations by building a binary search tree (BST) with n English words as keys and their French equivalents as satellite data.
- ❑ Because you will search the tree for each individual word in the text, you want the total time spent searching to be as low as possible.
- ❑ What you need is an *optimal binary search tree*. Formally, given a sequence $K = \langle k_1, k_2, \dots, k_n \rangle$ of n distinct keys, sorted ($k_1 < k_2 < \dots < k_n$).
 - Want to build a binary search trees from the keys.
 - For k_i , have probability p_i that a search is for k_i .
 - Want BST with minimum expected search cost.
 - Actual cost = number of items examined.

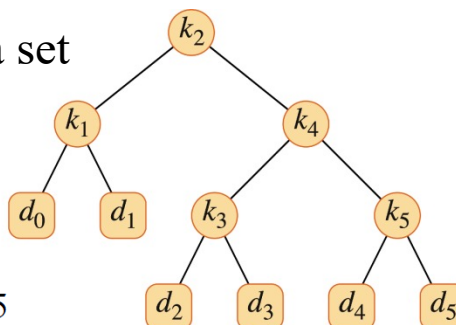
Optimal binary search trees

- Some searches may be for values not in K , you also have $n + 1$ “dummy” keys $d_0, d_1, d_2, \dots, d_n$ representing those values.
- In particular, d_0 represents all values less than k_1 , d_n represents all values greater than k_n , and for $i = 1, 2, \dots, n - 1$, the dummy key d_i represents all values between k_i and k_{i+1} .
- For each dummy key d_i , you have the probability q_i that a search corresponds to d_i .
- The figure in the next slide shows two binary search trees for a set of $n = 5$ keys.
- Each key k_i is an internal node, and each dummy key d_i is a leaf.
- Since every search is either successful (finding some key k_i), or unsuccessful (finding some dummy key d_i), we have

$$\sum_{i=1}^n p_i + \sum_{i=0}^n q_i = 1 .$$

Optimal binary search trees

Two binary search trees for a set of $n = 5$ keys with the following probabilities.



i	0	1	2	3	4	5
p_i		0.15	0.10	0.05	0.10	0.20
q_i	0.05	0.10	0.05	0.05	0.05	0.10

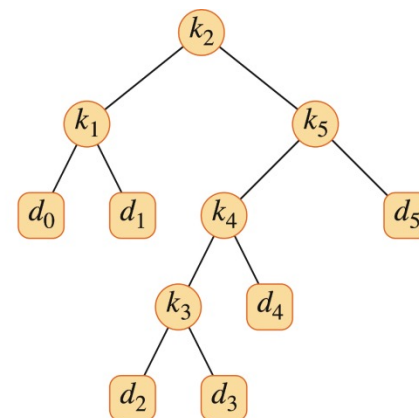
(a) A binary search tree with expected search cost 2.80.

(b) A binary search tree with expected search cost 2.75.

This tree is optimal.

node	depth	probability	contribution
k_1	1	0.15	0.30
k_2	0	0.10	0.10
k_3	2	0.05	0.15
k_4	1	0.10	0.20
k_5	2	0.20	0.60
d_0	2	0.05	0.15
d_1	2	0.10	0.30
d_2	3	0.05	0.20
d_3	3	0.05	0.20
d_4	3	0.05	0.20
d_5	3	0.10	0.40
Total			2.80

(a)



node	depth	probability	contribution
k_1	1	0.15	0.30
k_2	0	0.10	0.10
k_3	3	0.05	0.20
k_4	2	0.10	0.30
k_5	1	0.20	0.40
d_0	2	0.05	0.15
d_1	2	0.10	0.30
d_2	4	0.05	0.25
d_3	4	0.05	0.25
d_4	3	0.05	0.20
d_5	2	0.10	0.30
Total			2.75

(b)

Optimal binary search trees

- Knowing the probabilities of searches for each key and each dummy key allows us to determine the expected cost of a search in a given binary search tree T .
- Let us assume that the actual cost of a search equals the number of nodes examined, which is the depth of the node found by the search in T , plus 1. Thus, for key k_i , $\text{cost} = \text{depth}_T(k_i) + 1$, where $\text{depth}_T(k_i)$ equals the depth of k_i in BST T .
- Then the expected cost of a search in T is

$$\begin{aligned} E[\text{search cost in } T] &= \sum_{i=1}^n (\text{depth}_T(k_i) + 1) \cdot p_i + \sum_{i=0}^n (\text{depth}_T(d_i) + 1) \cdot q_i \\ &= 1 + \sum_{i=1}^n \text{depth}_T(k_i) \cdot p_i + \sum_{i=0}^n \text{depth}_T(d_i) \cdot q_i , \end{aligned}$$

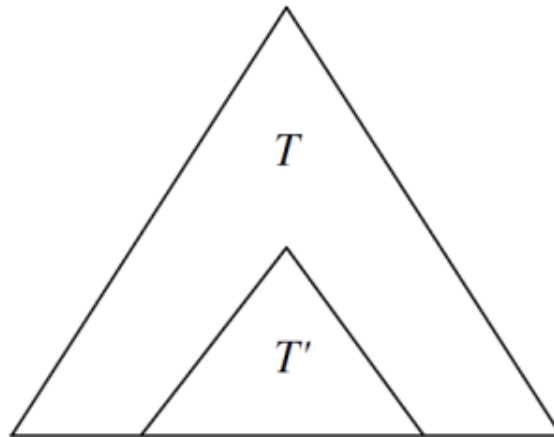
Optimal binary search trees

- ❑ *Observations* from the figure
 - Optimal BST might not have smallest height.
 - Optimal BST might not have highest-probability key at root.
- ❑ Build by exhaustive checking?
 - Construct each n -node BST.
 - For each, put in keys.
 - Then compute expected search cost.
 - But there are $\Omega(4^n/n^{3/2})$ different BSTs with n nodes.
- ❑ We will solve this problem more efficiently with dynamic programming.

Optimal binary search trees

□ Step 1: The structure of an optimal binary search tree

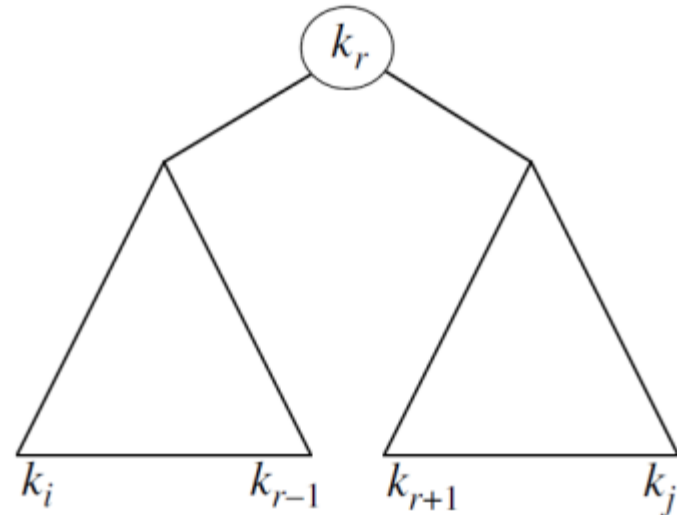
Consider any subtree of a BST. It contains keys in a contiguous range $k_i, \dots, k_j$ and its leaves the dummy keys $d_{i-1}, \dots, d_j$ for some $1 \leq i \leq j \leq n$.



If T is an optimal BST and T contains subtree T' with keys $k_i, \dots, k_j$, then T' must be an optimal BST for keys $k_i, \dots, k_j$ and dummy keys $d_{i-1}, \dots, d_j$.

Optimal binary search trees

- Use optimal substructure to construct an optimal solution to the problem from optimal solutions to subproblems:
 - Given keys $k_i, \dots, k_j$ (the problem)
 - One of them, k_r , where $i \leq r \leq j$, must be the root.
 - Left subtree of k_r contains $k_i, \dots, k_{r-1}$ and dummy keys $d_{i-1}, \dots, d_{r-1}$.
 - Right subtree of k_r contains $k_{r+1}, \dots, k_j$ and dummy keys $d_r, \dots, d_j$.
- If
 - you examine all candidate roots k_r , for $i \leq r \leq j$, and
 - you determine all optimal BSTs containing $k_i, \dots, k_{r-1}$ and containing $k_{r+1}, \dots, k_j$,then you are guaranteed to find an optimal BST for $k_i, \dots, k_j$.



Optimal binary search trees

□ Step 2: A recursive solution

Subproblem domain:

- Find optimal BST for $k_i, \dots, k_j$, where $i \geq 1, j \leq n, j \geq i - 1$.
- When $j = i - 1$, the tree contains the dummy keys $d_{i-1}, \dots, d_j$.
- Define $e[i, j]$ = expected search cost of optimal BST for $k_i, \dots, k_j$.
- If $j = i - 1$, then $e[i, i - 1] = q_{i-1}$
- If $j \geq i$,
 - Select a root k_r , for some $i \leq r \leq j$.
 - Make an optimal BST with $k_i, \dots, k_{r-1}$ as the left subtree.
 - Make an optimal BST with $k_{r+1}, \dots, k_j$ as the right subtree.

Optimal binary search trees

- When a subtree becomes a subtree of a node:
 - Depth of every node in subtree goes up by 1.
 - Expected search cost increases by the sum of all the probabilities in the subtree:

$$w(i, j) = \sum_{l=i}^j p_l + \sum_{l=i-1}^j q_l .$$

- If k_r is the root of an optimal BST for $k_i, \dots, k_j$:

$$e[i, j] = p_r + (e[i, r-1] + w(i, r-1)) + (e[r+1, j] + w(r+1, j)) .$$

- Note that $w(i, j) = w(i, r-1) + p_r + w(r+1, j)$,

- Therefore,

$$e[i, j] = e[i, r-1] + e[r+1, j] + w(i, j) .$$

- This equation assumes that we already know which key is k_r ,₁₂ but we do not.

Optimal binary search trees

- Try all candidates, and pick the root that gives the lowest expected search cost, giving the final recursive formulation:

$$e[i, j] = \begin{cases} q_{i-1} & \text{if } j = i - 1, \\ \min_{i \leq r \leq j} \{e[i, r-1] + e[r+1, j] + w(i, j)\} & \text{if } i \leq j. \end{cases}$$

- The $e[i, j]$ values give the expected search costs in optimal binary search trees.
- To help keep track of the structure of optimal binary search trees, define $root[i, j]$, for $1 \leq i \leq j \leq n$, to be the index r for which k_r is the root of an optimal binary search tree containing keys $k_i, \dots, k_j$.

Optimal binary search trees

□ Step 3: Computing the expected search cost of an optimal binary search tree

We store the values in a table:

$$e[\underbrace{1:n+1}_{\text{can store}}, \underbrace{0:n}_{\text{can store}}]$$
$$e[n+1, n] \quad e[1, 0]$$

- Will use only entries $e[i, j]$, where $j \geq i - 1$.
- Will also compute
 $root[i, j] = \text{root of subtree with keys } k_i, \dots, k_j, \text{ for } 1 \leq i \leq j \leq n.$
- One other table: do not recompute $w(i, j)$ from scratch every time we need it. (Would take $\Theta(j - i)$ additions.)
- Table $w[1 : n + 1, 0 : n]$
- For the base case, compute $w[i, i - 1] = q_{i-1}$ for $1 \leq i \leq n + 1$.
- For $j \geq i$, compute $w[i, j] = w[i, j - 1] + p_j + q_j$.
- Can compute all $\Theta(n^2)$ values in $O(1)$ time each.

Optimal binary search trees

- The OPTIMAL-BST procedure takes as inputs the probabilities $p_1, \dots, p_n$, and $q_0, \dots, q_n$ and the size n , and it returns the tables e and $root$.

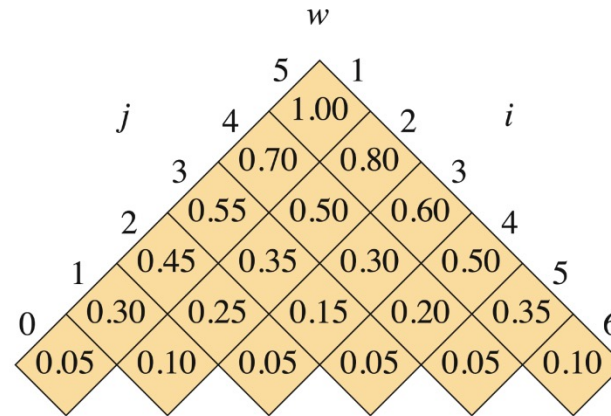
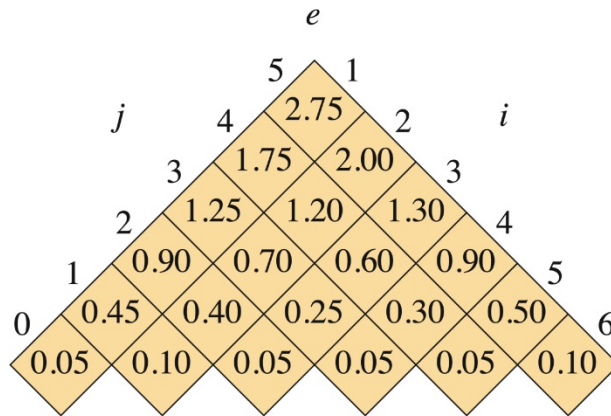
OPTIMAL-BST(p, q, n)

```
1  let  $e[1:n+1, 0:n]$ ,  $w[1:n+1, 0:n]$ ,  
    and  $root[1:n, 1:n]$  be new tables  
2  for  $i = 1$  to  $n+1$            // base cases  
3       $e[i, i-1] = q_{i-1}$       // equation (14.14)  
4       $w[i, i-1] = q_{i-1}$   
5  for  $l = 1$  to  $n$   
6      for  $i = 1$  to  $n-l+1$   
7           $j = i+l-1$   
8           $e[i, j] = \infty$   
9           $w[i, j] = w[i, j-1] + p_j + q_j$            // equation (14.15)  
10         for  $r = i$  to  $j$            // try all possible roots  $r$   
11              $t = e[i, r-1] + e[r+1, j] + w[i, j]$  // equation (14.14)  
12             if  $t < e[i, j]$            // new minimum?  
13                  $e[i, j] = t$   
14                  $root[i, j] = r$   
15  return  $e$  and  $root$ 
```

Optimal binary search trees

- The following $e[i, j]$, $w[i, j]$, and $root[i, j]$ tables are rotated so that the diagonals run horizontally.

i	0	1	2	3	4	5
p_i		0.15	0.10	0.05	0.10	0.20
q_i	0.05	0.10	0.05	0.05	0.05	0.10



k_2 is the root

k_1 is the left child of k_2

d_0 is the left child of k_1

d_1 is the right child of k_1

k_5 is the right child of k_2

k_4 is the left child of k_5

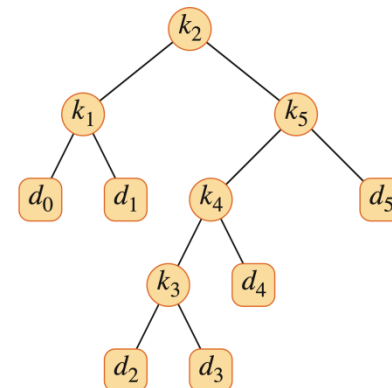
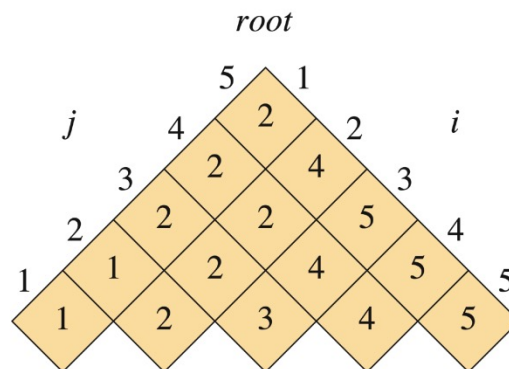
k_3 is the left child of k_4

d_2 is the left child of k_3

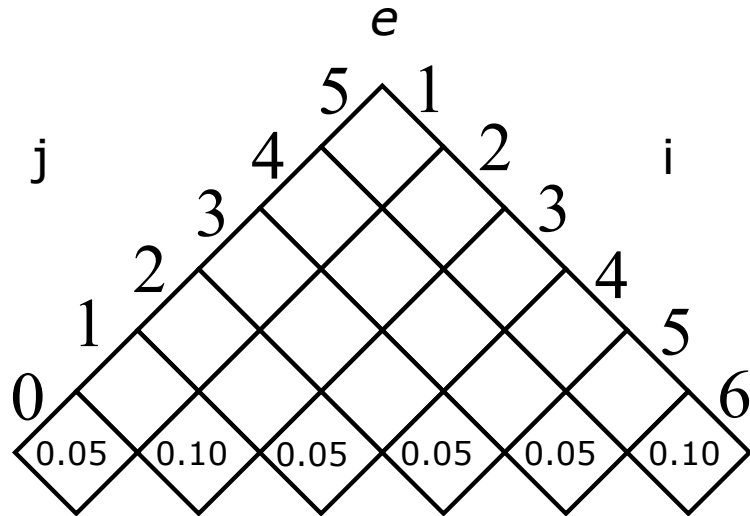
d_3 is the right child of k_3

d_4 is the right child of k_4

d_5 is the right child of k_5



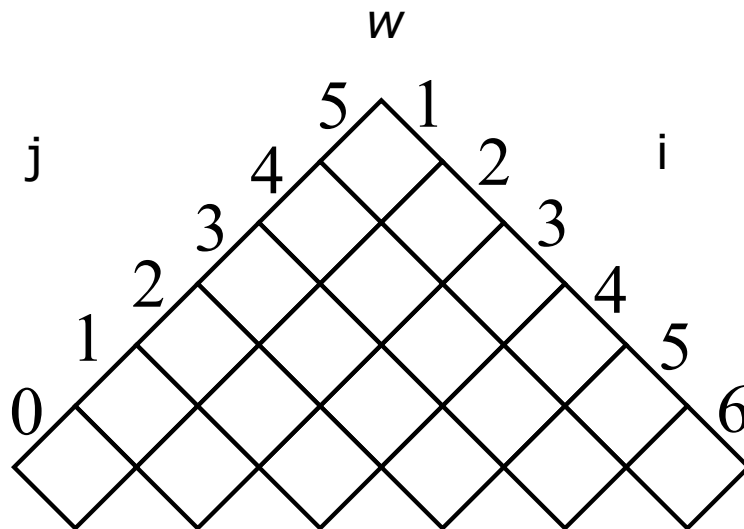
Optimal binary search trees



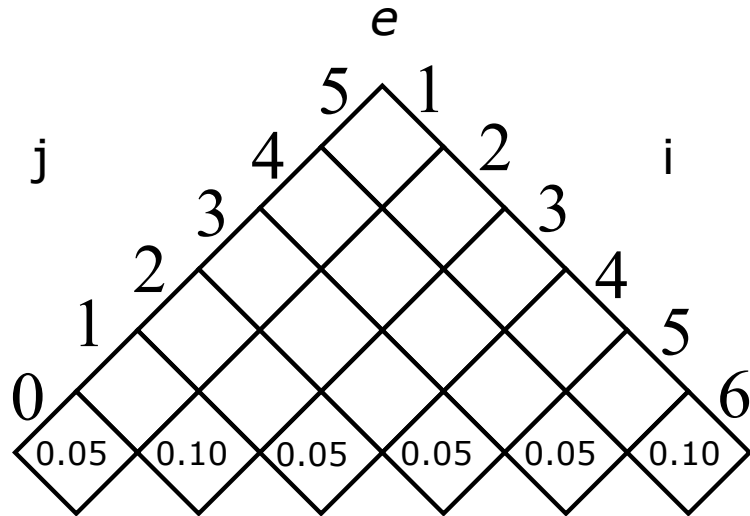
i	0	1	2	3	4	5
p_i		0.15	0.10	0.05	0.10	0.20
q_i	0.05	0.10	0.05	0.05	0.05	0.10

$$e[i, i - 1] = q_{i-1}$$

$$w[i, i - 1] = q_{i-1}$$



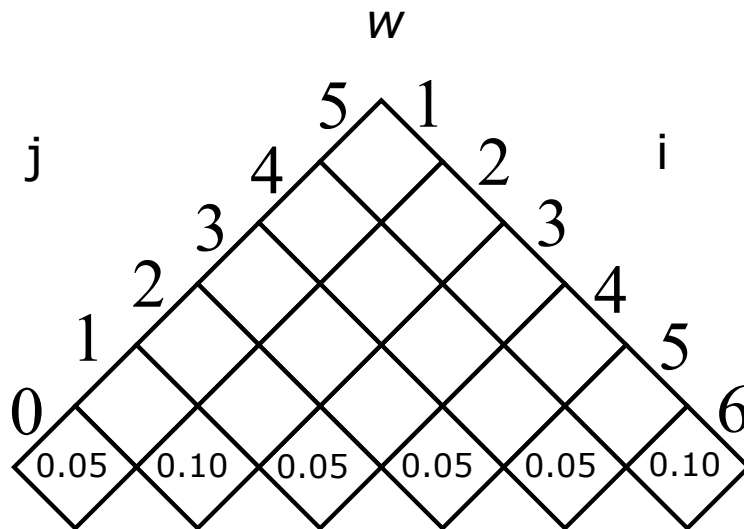
Optimal binary search trees



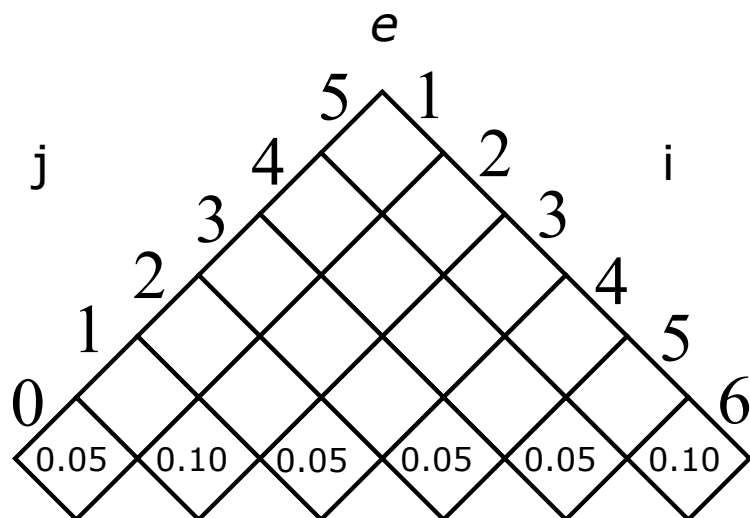
i	0	1	2	3	4	5
p_i		0.15	0.10	0.05	0.10	0.20
q_i	0.05	0.10	0.05	0.05	0.05	0.10

$$e[i, i - 1] = q_{i-1}$$

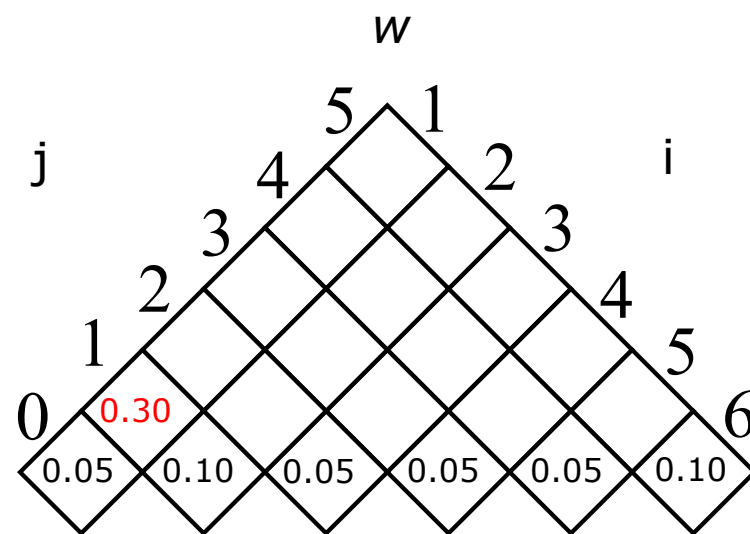
$$w[i, i - 1] = q_{i-1}$$



Optimal binary search trees



i	0	1	2	3	4	5
p_i		0.15	0.10	0.05	0.10	0.20
q_i	0.05	0.10	0.05	0.05	0.05	0.10



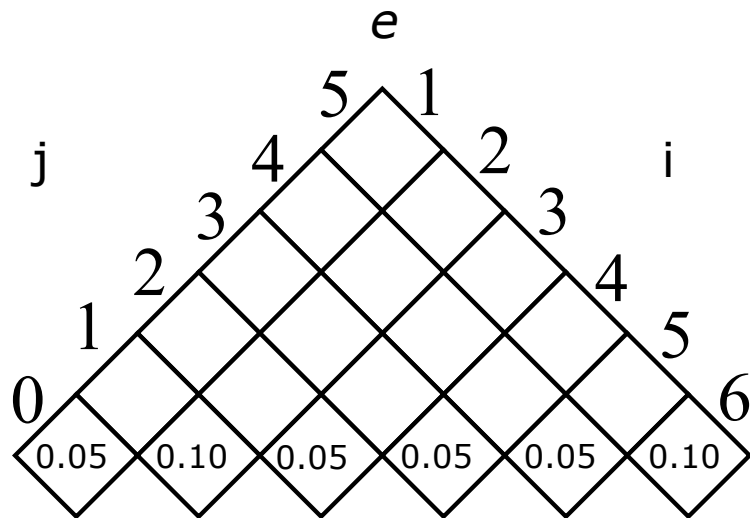
$$w[i, j] = w[i, j - 1] + p_j + q_j$$

$$W[1,1] = w[1,0] + p_1 + q_1$$

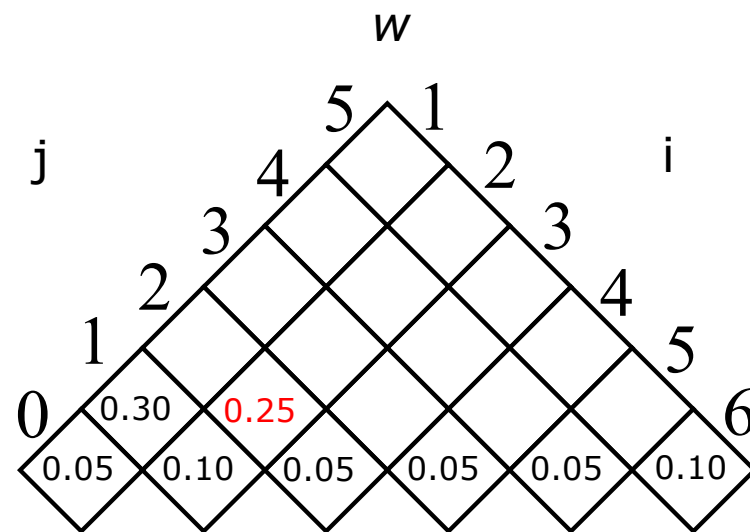
$$W[1,1] = 0.05 + 0.15 + 0.10$$

$$W[1,1] = 0.30$$

Optimal binary search trees



i	0	1	2	3	4	5
p_i		0.15	0.10	0.05	0.10	0.20
q_i	0.05	0.10	0.05	0.05	0.05	0.10



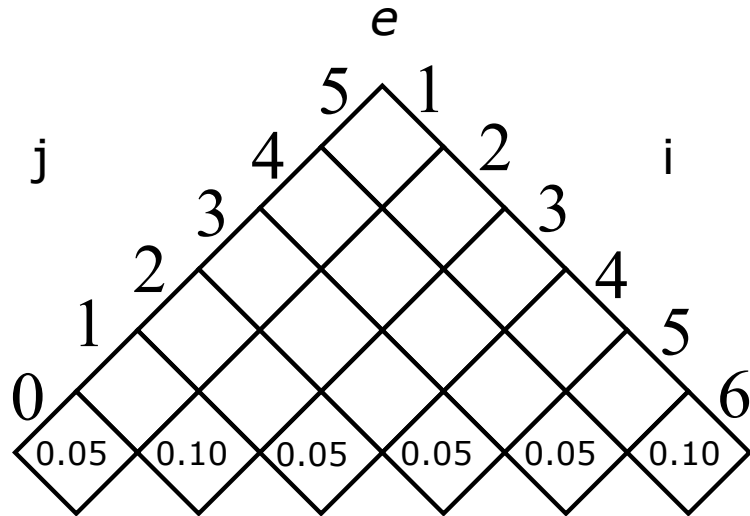
$$w[i, j] = w[i, j - 1] + p_j + q_j$$

$$W[2,2] = w[2,1] + p_2 + q_2$$

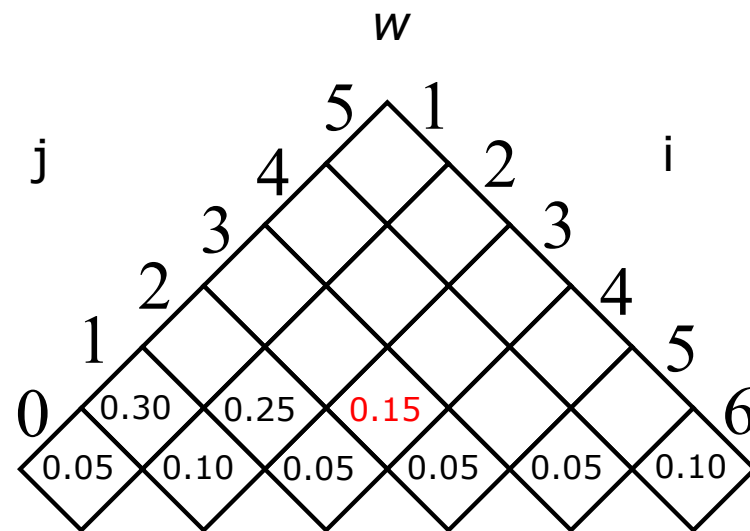
$$W[2,2] = 0.10 + 0.10 + 0.05$$

$$W[2,2] = 0.25$$

Optimal binary search trees



i	0	1	2	3	4	5
p_i		0.15	0.10	0.05	0.10	0.20
q_i	0.05	0.10	0.05	0.05	0.05	0.10



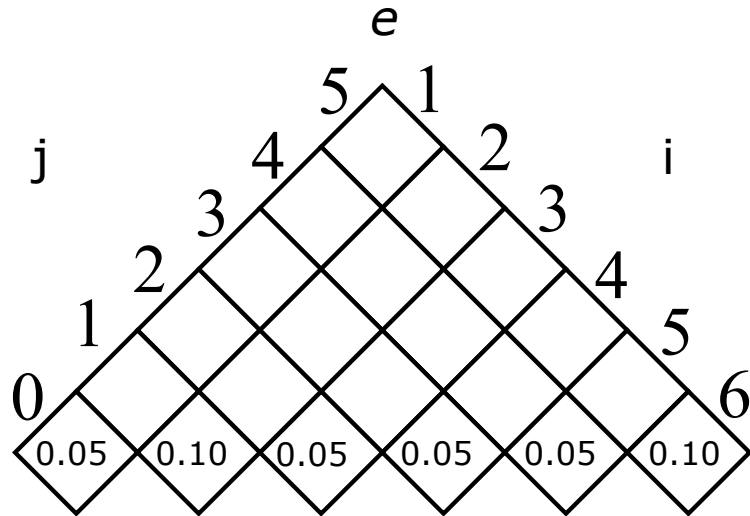
$$w[i, j] = w[i, j - 1] + p_j + q_j$$

$$W[3,3] = w[3,2] + p_3 + q_3$$

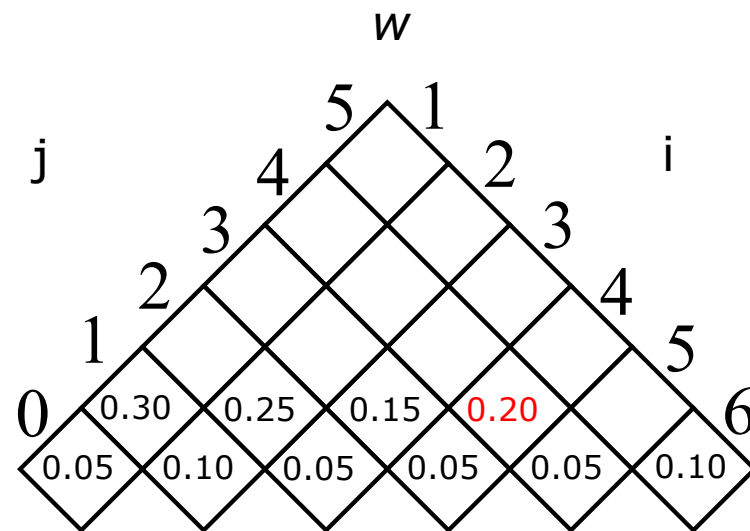
$$W[3,3] = 0.05 + 0.05 + 0.05$$

$$W[3,3] = 0.15$$

Optimal binary search trees



i	0	1	2	3	4	5
p_i		0.15	0.10	0.05	0.10	0.20
q_i	0.05	0.10	0.05	0.05	0.05	0.10



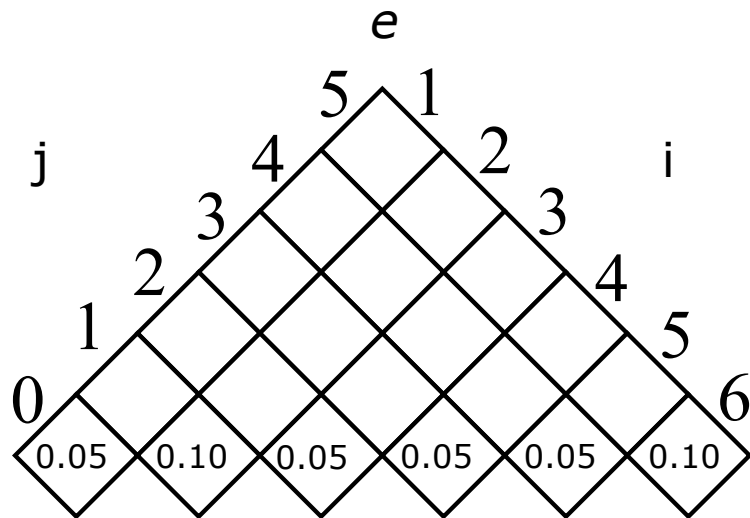
$$w[i, j] = w[i, j - 1] + p_j + q_j$$

$$W[4,4] = w[4,3] + p_4 + q_4$$

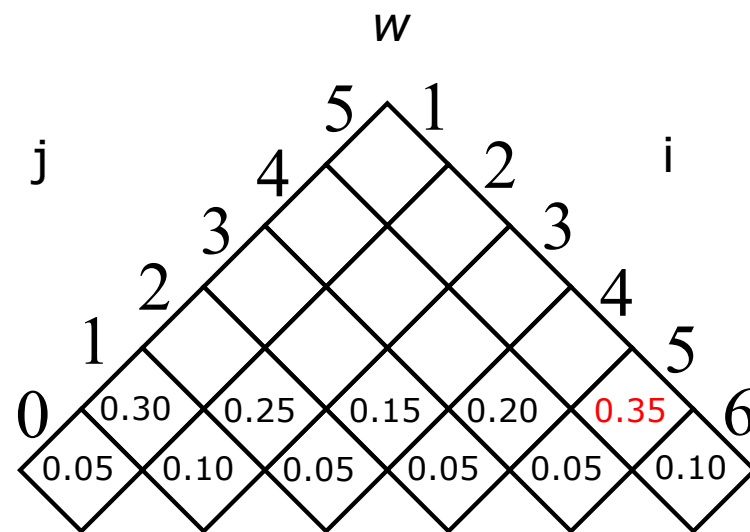
$$W[4,4] = 0.05 + 0.10 + 0.05$$

$$W[4,4] = 0.20$$

Optimal binary search trees



i	0	1	2	3	4	5
p_i		0.15	0.10	0.05	0.10	0.20
q_i	0.05	0.10	0.05	0.05	0.05	0.10



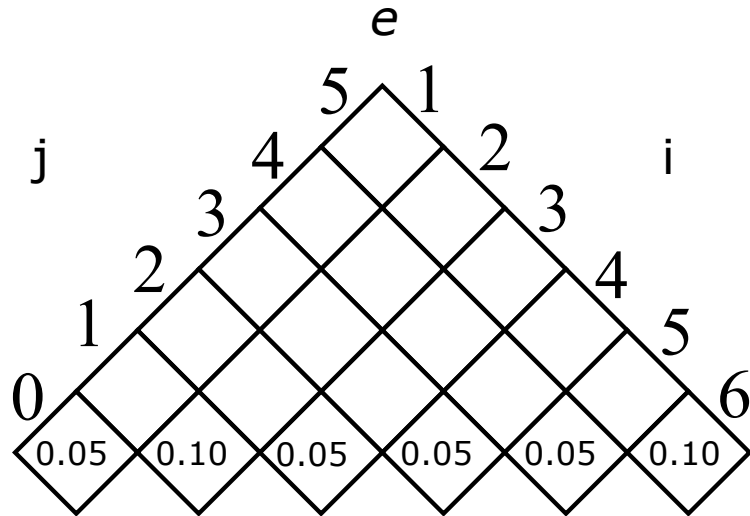
$$w[i, j] = w[i, j - 1] + p_j + q_j$$

$$W[5,5] = w[5,4] + p_5 + q_5$$

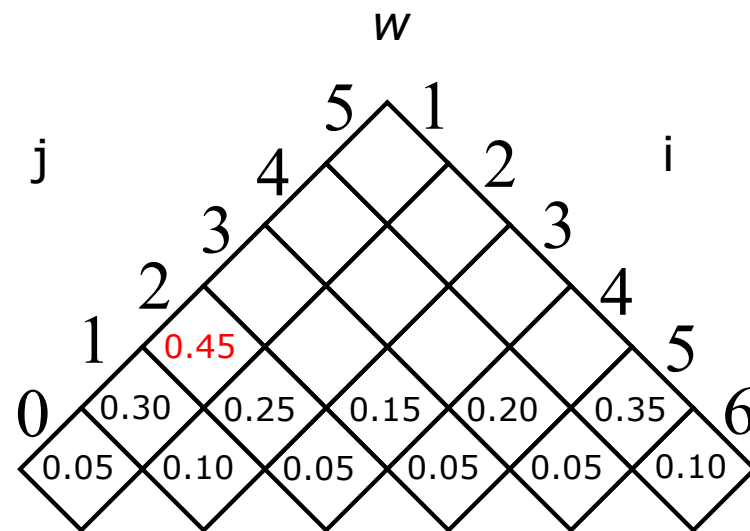
$$W[5,5] = 0.05 + 0.20 + 0.10$$

$$W[5,5] = 0.35$$

Optimal binary search trees



i	0	1	2	3	4	5
p_i		0.15	0.10	0.05	0.10	0.20
q_i	0.05	0.10	0.05	0.05	0.05	0.10



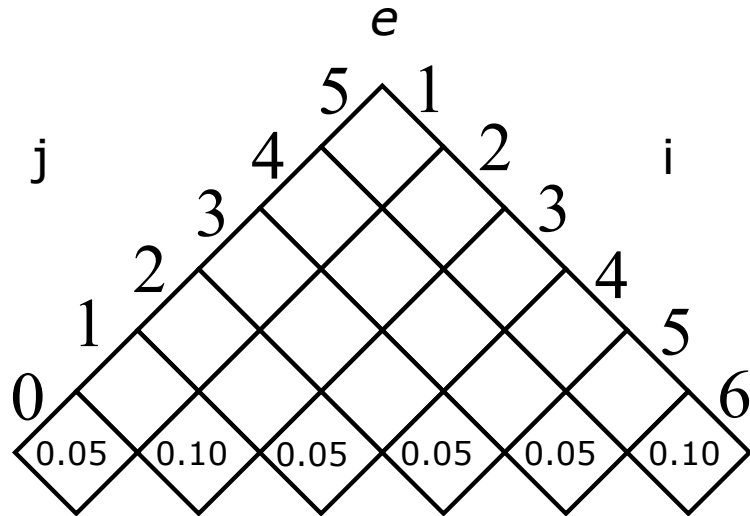
$$w[i, j] = w[i, j - 1] + p_j + q_j$$

$$W[1, 2] = w[1, 1] + p_2 + q_2$$

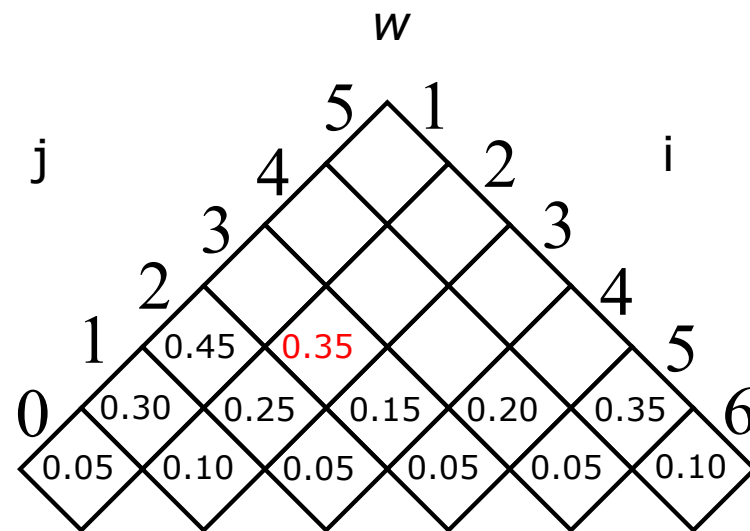
$$W[1, 2] = 0.30 + 0.10 + 0.05$$

$$W[1, 2] = 0.45$$

Optimal binary search trees



i	0	1	2	3	4	5
p_i		0.15	0.10	0.05	0.10	0.20
q_i	0.05	0.10	0.05	0.05	0.05	0.10



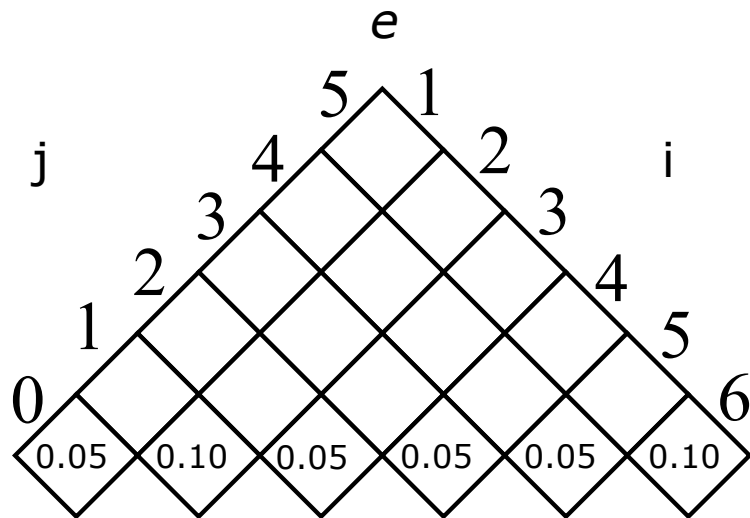
$$w[i, j] = w[i, j - 1] + p_j + q_j$$

$$W[2,3] = w[2,2] + p_3 + q_3$$

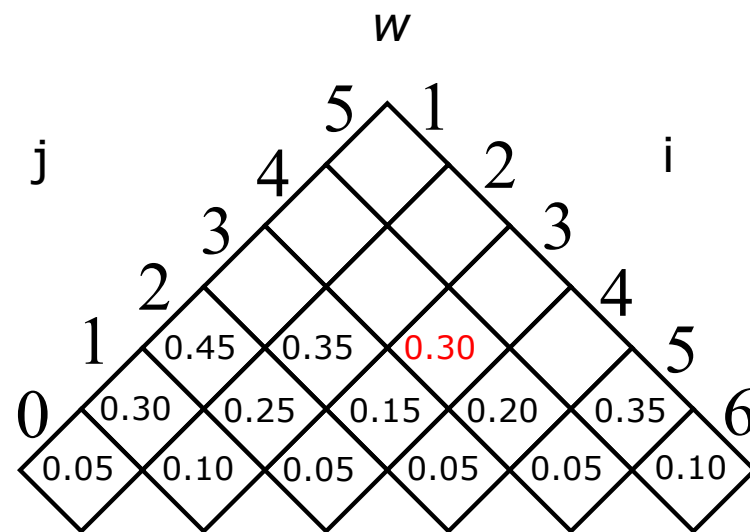
$$W[2,3] = 0.25 + 0.05 + 0.05$$

$$W[2,3] = 0.35$$

Optimal binary search trees



i	0	1	2	3	4	5
p_i		0.15	0.10	0.05	0.10	0.20
q_i	0.05	0.10	0.05	0.05	0.05	0.10



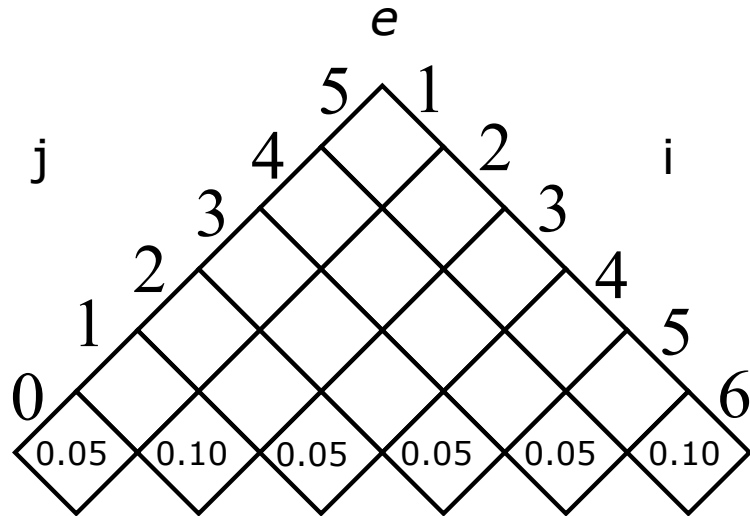
$$w[i, j] = w[i, j - 1] + p_j + q_j$$

$$W[3,4] = w[3,3] + p_4 + q_4$$

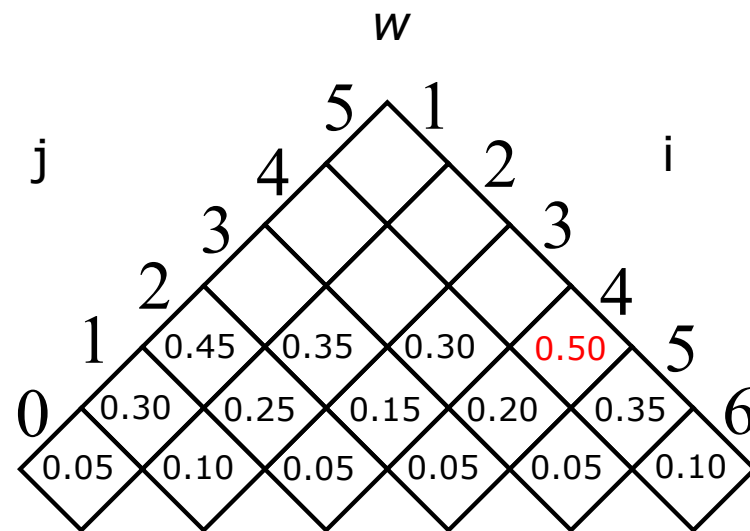
$$W[3,4] = 0.15 + 0.10 + 0.05$$

$$W[3,4] = 0.30$$

Optimal binary search trees



i	0	1	2	3	4	5
p_i		0.15	0.10	0.05	0.10	0.20
q_i	0.05	0.10	0.05	0.05	0.05	0.10



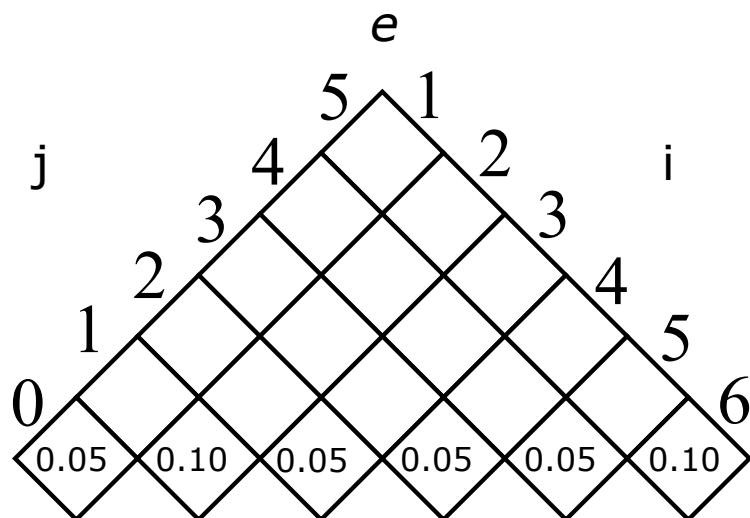
$$w[i, j] = w[i, j - 1] + p_j + q_j$$

$$W[4, 5] = w[4, 4] + p_5 + q_5$$

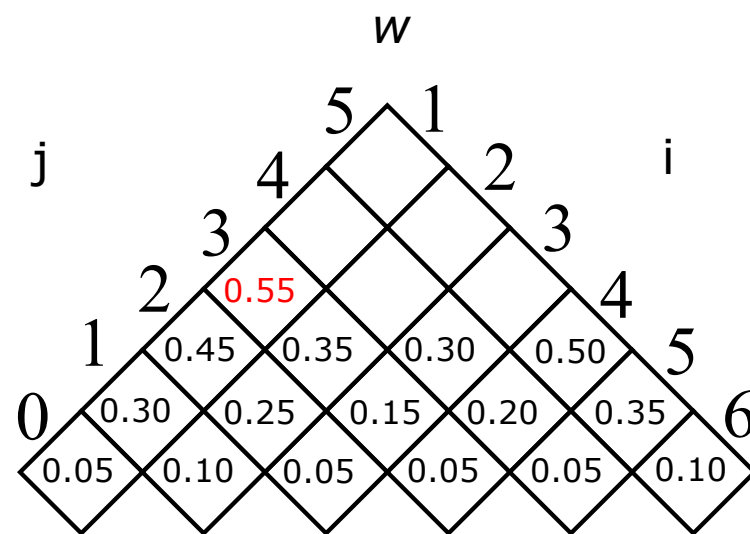
$$W[3, 4] = 0.20 + 0.20 + 0.10$$

$$W[3, 4] = 0.50$$

Optimal binary search trees



i	0	1	2	3	4	5
p_i		0.15	0.10	0.05	0.10	0.20
q_i	0.05	0.10	0.05	0.05	0.05	0.10



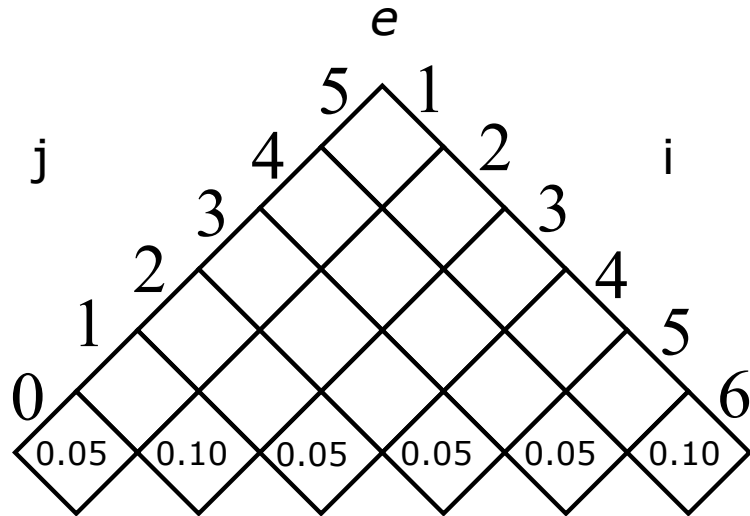
$$w[i, j] = w[i, j - 1] + p_j + q_j$$

$$W[1, 3] = w[1, 2] + p_3 + q_3$$

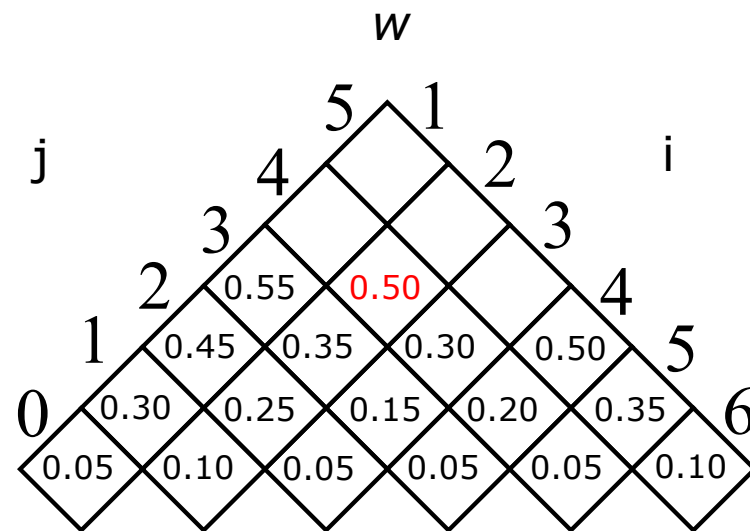
$$W[1, 3] = 0.45 + 0.05 + 0.05$$

$$W[1, 3] = 0.55$$

Optimal binary search trees



i	0	1	2	3	4	5
p_i		0.15	0.10	0.05	0.10	0.20
q_i	0.05	0.10	0.05	0.05	0.05	0.10



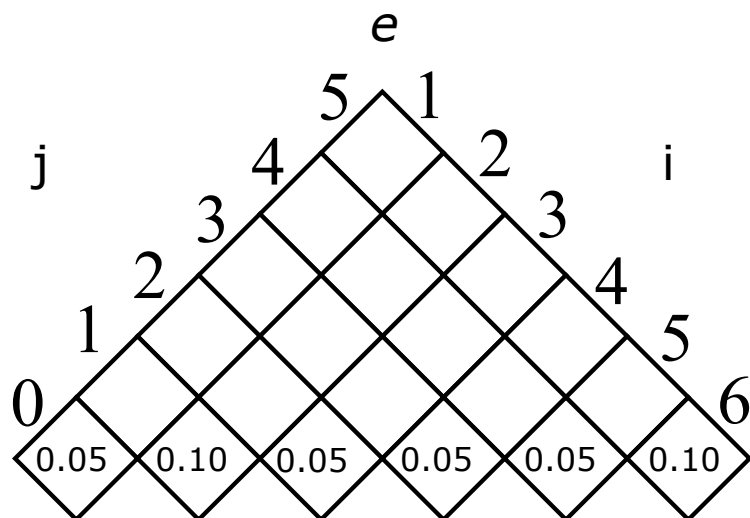
$$w[i, j] = w[i, j - 1] + p_j + q_j$$

$$W[2,4] = w[2,3] + p_4 + q_4$$

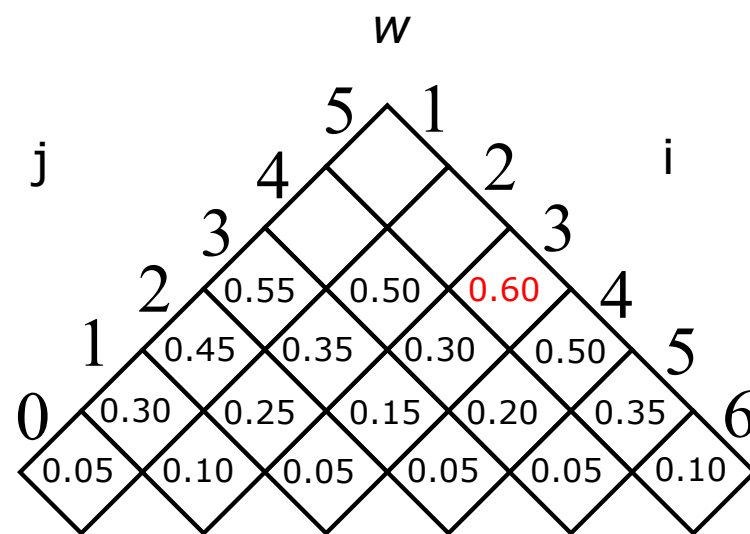
$$W[2,4] = 0.35 + 0.10 + 0.05$$

$$W[1,3] = 0.50$$

Optimal binary search trees



i	0	1	2	3	4	5
p_i		0.15	0.10	0.05	0.10	0.20
q_i	0.05	0.10	0.05	0.05	0.05	0.10



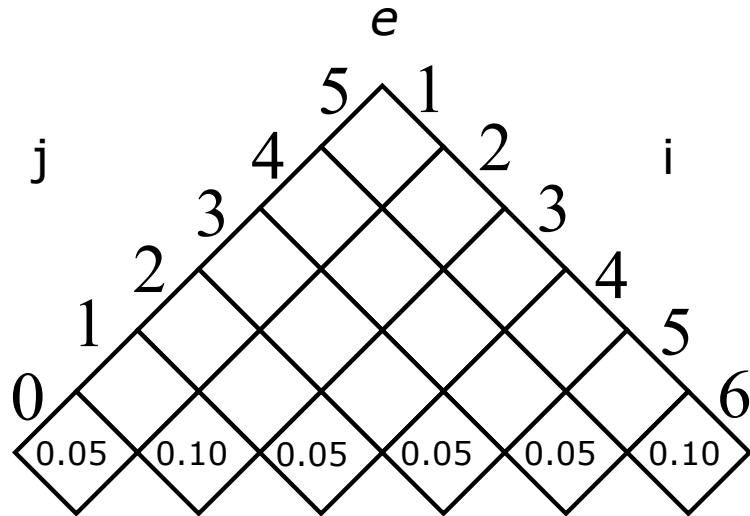
$$w[i, j] = w[i, j - 1] + p_j + q_j$$

$$W[3, 5] = w[3, 4] + p_5 + q_5$$

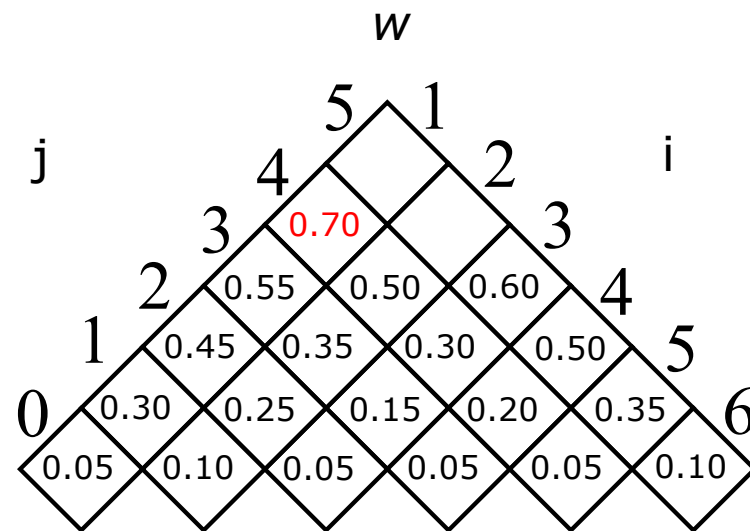
$$W[3, 5] = 0.30 + 0.20 + 0.10$$

$$W[1, 3] = 0.60$$

Optimal binary search trees



i	0	1	2	3	4	5
p_i		0.15	0.10	0.05	0.10	0.20
q_i	0.05	0.10	0.05	0.05	0.05	0.10



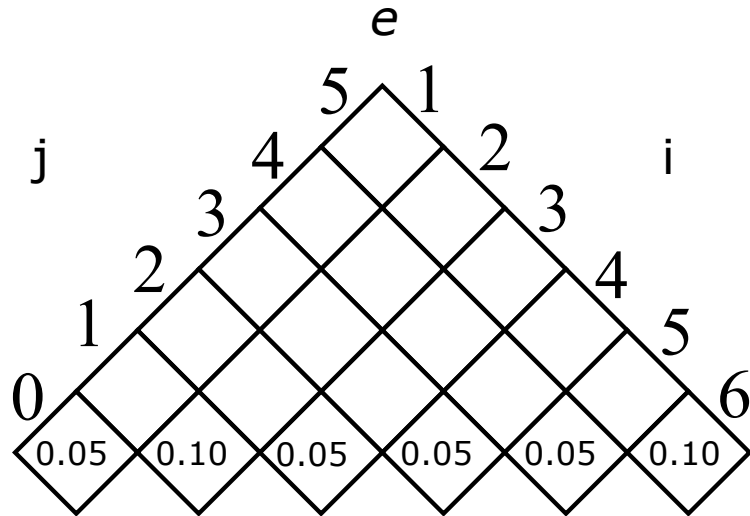
$$w[i, j] = w[i, j - 1] + p_j + q_j$$

$$W[1,4] = w[1,3] + p_4 + q_4$$

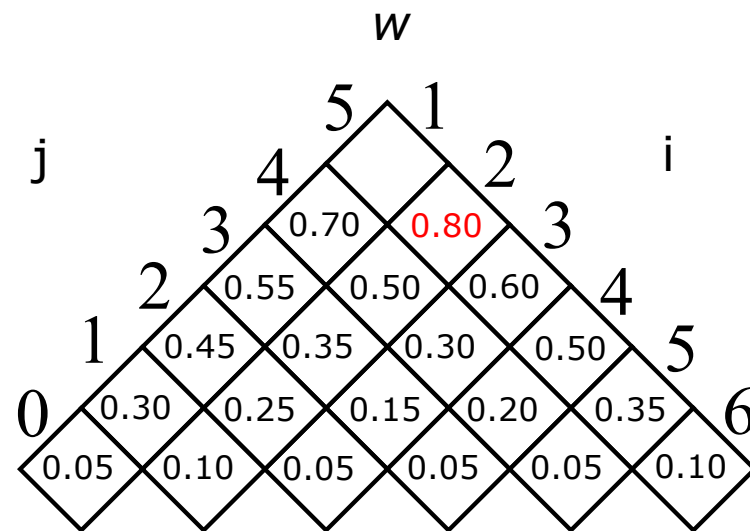
$$W[1,4] = 0.55 + 0.10 + 0.05$$

$$W[1,4] = 0.70$$

Optimal binary search trees



i	0	1	2	3	4	5
p_i		0.15	0.10	0.05	0.10	0.20
q_i	0.05	0.10	0.05	0.05	0.05	0.10



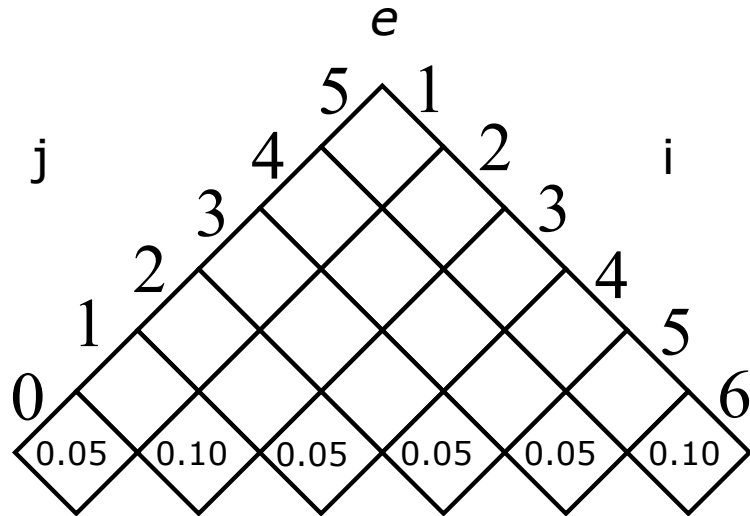
$$w[i, j] = w[i, j - 1] + p_j + q_j$$

$$W[2, 5] = w[2, 4] + p_5 + q_5$$

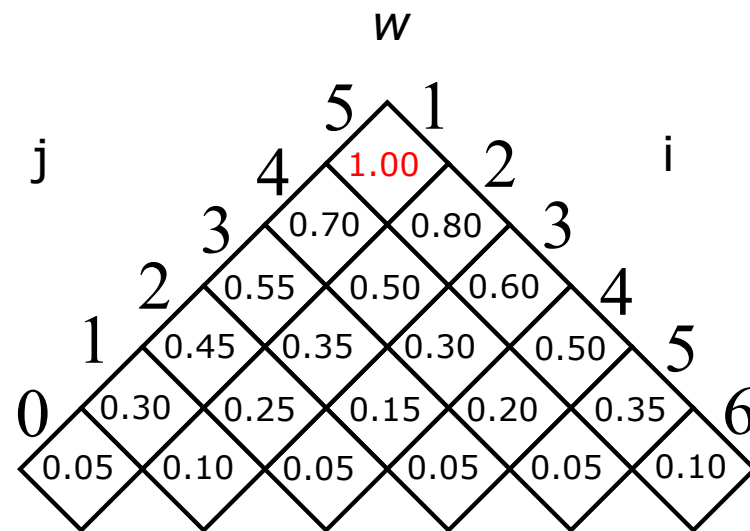
$$W[2, 5] = 0.50 + 0.20 + 0.10$$

$$W[2, 5] = 0.80$$

Optimal binary search trees



i	0	1	2	3	4	5
p_i		0.15	0.10	0.05	0.10	0.20
q_i	0.05	0.10	0.05	0.05	0.05	0.10



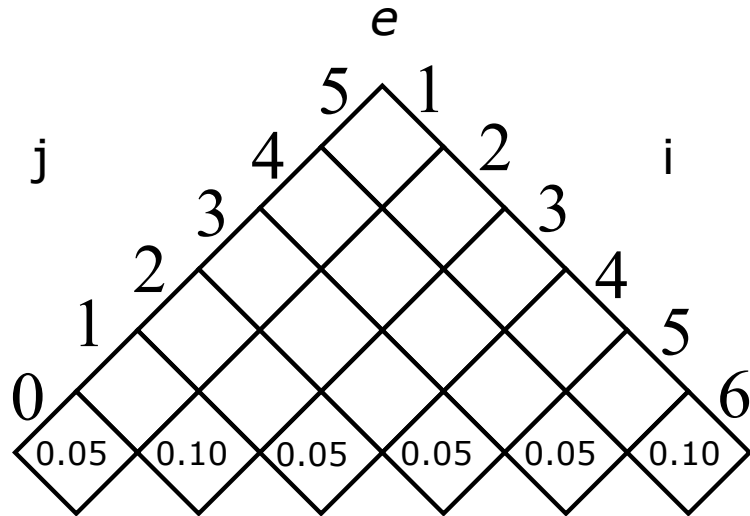
$$w[i, j] = w[i, j - 1] + p_j + q_j$$

$$W[1, 5] = w[1, 4] + p_5 + q_5$$

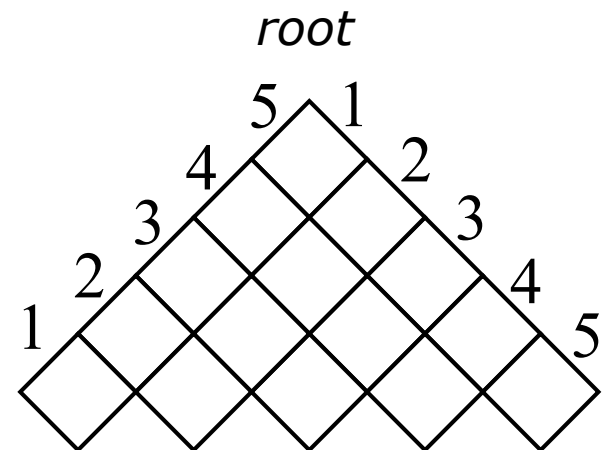
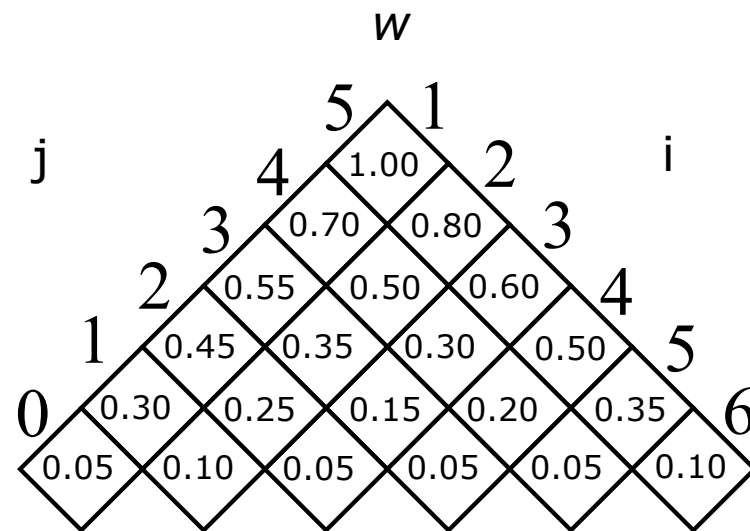
$$W[1, 5] = 0.70 + 0.20 + 0.10$$

$$W[1, 5] = 1.00$$

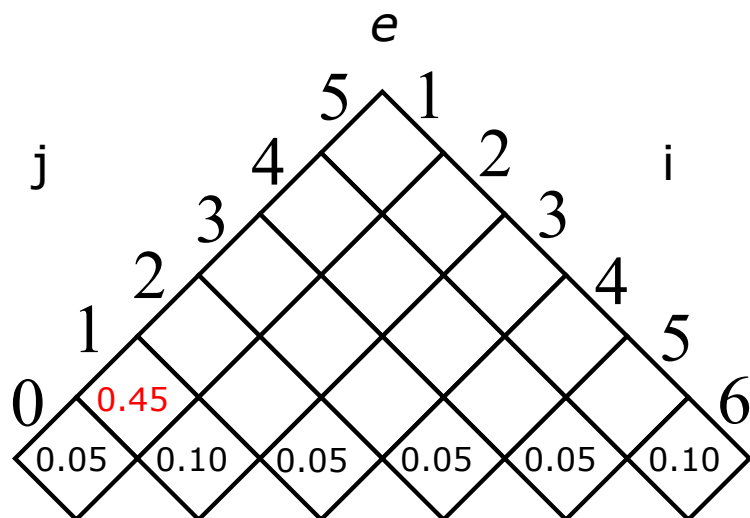
Optimal binary search trees



i	0	1	2	3	4	5
p_i		0.15	0.10	0.05	0.10	0.20
q_i	0.05	0.10	0.05	0.05	0.05	0.10



Optimal binary search trees



i	0	1	2	3	4	5
p_i		0.15	0.10	0.05	0.10	0.20
q_i	0.05	0.10	0.05	0.05	0.05	0.10

$$e[i, j] = e[i, r - 1] + e[r + 1, j] + w(i, j)$$

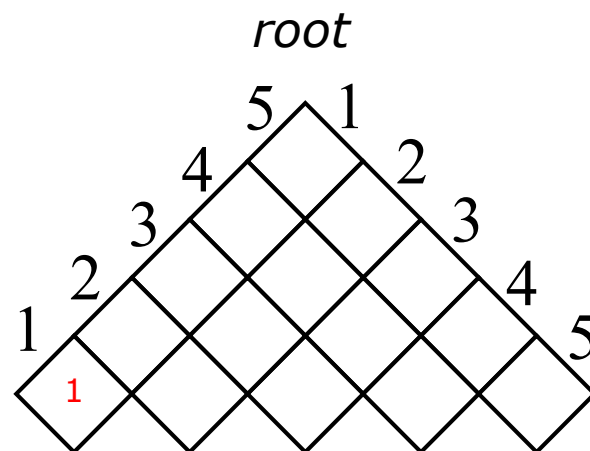
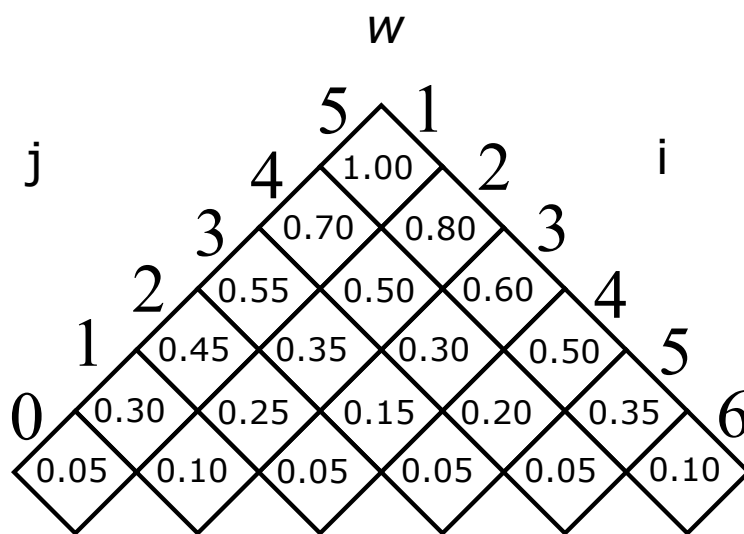
$$i \leq r \leq j$$

$r=1$

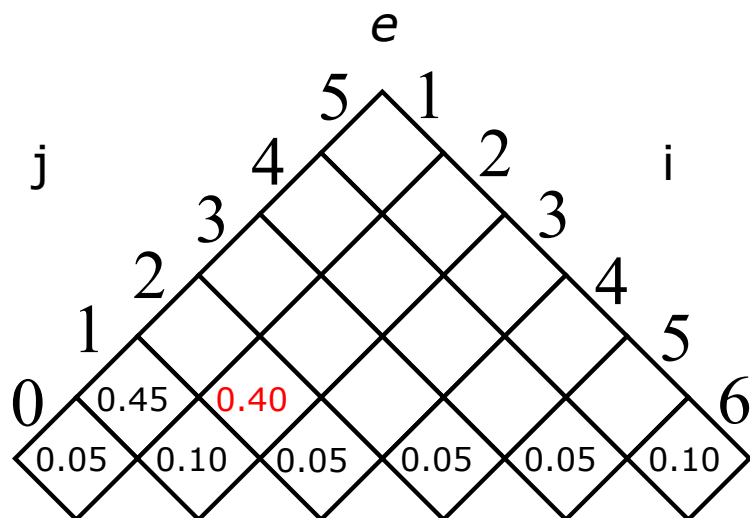
$$e[1, 1] = e[1, 0] + e[2, 1] + w(1, 1)$$

$$e[1, 1] = 0.05 + 0.10 + 0.30$$

$$e[1, 1] = 0.45$$



Optimal binary search trees



i	0	1	2	3	4	5
p_i		0.15	0.10	0.05	0.10	0.20
q_i	0.05	0.10	0.05	0.05	0.05	0.10

$$e[i, j] = e[i, r - 1] + e[r + 1, j] + w(i, j)$$

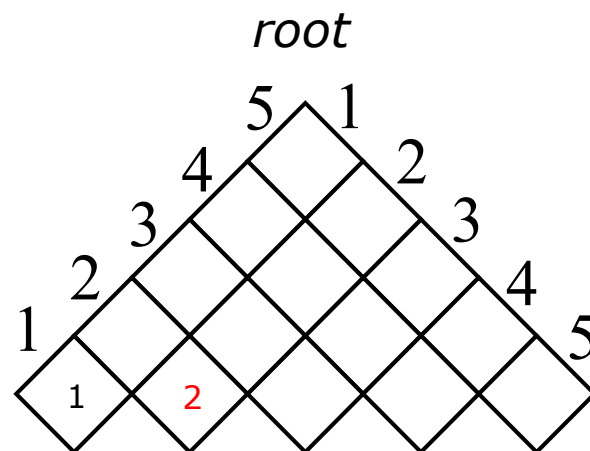
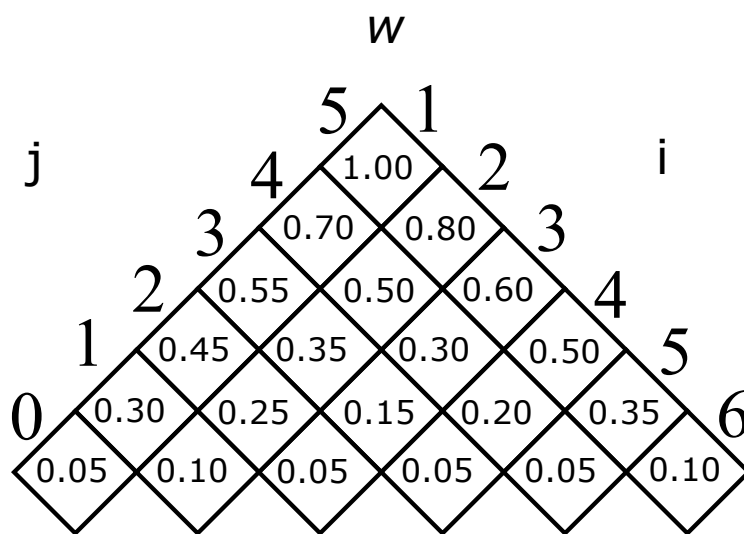
$$i \leq r \leq j$$

$$r=2$$

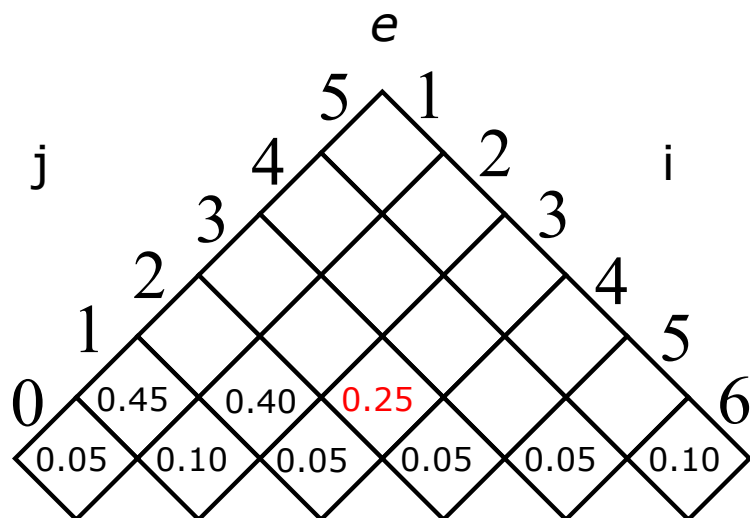
$$e[2,2] = e[2,1] + e[3,2] + w(2,2)$$

$$e[1,1] = 0.10 + 0.05 + 0.25$$

$$e[1,1] = 0.40$$



Optimal binary search trees



i	0	1	2	3	4	5
p_i		0.15	0.10	0.05	0.10	0.20
q_i	0.05	0.10	0.05	0.05	0.05	0.10

$$e[i, j] = e[i, r - 1] + e[r + 1, j] + w(i, j)$$

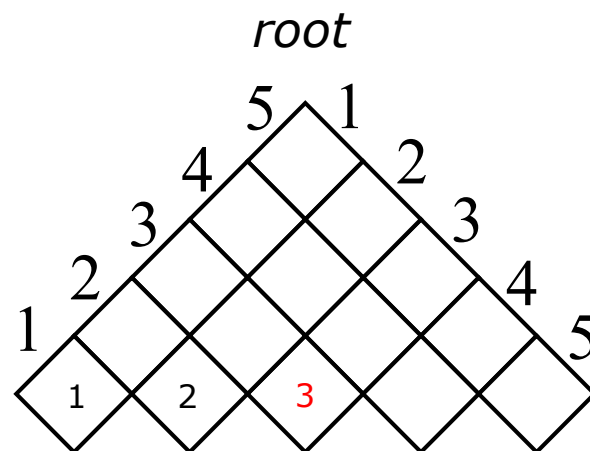
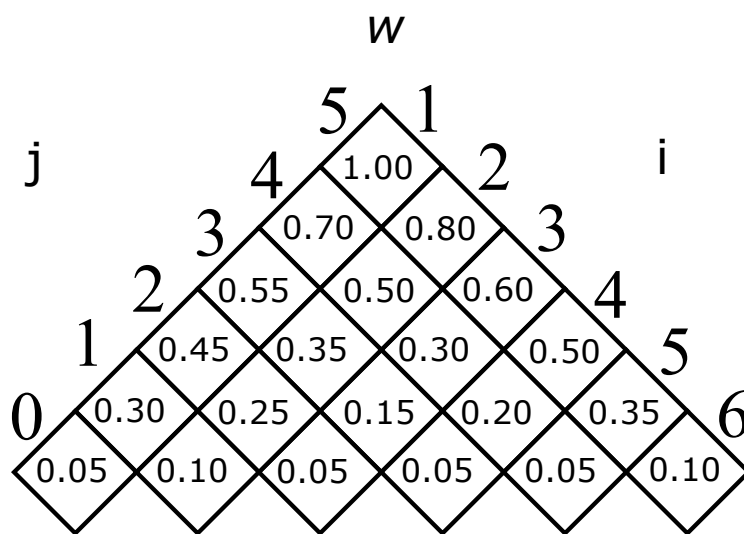
$$i \leq r \leq j$$

$$r=3$$

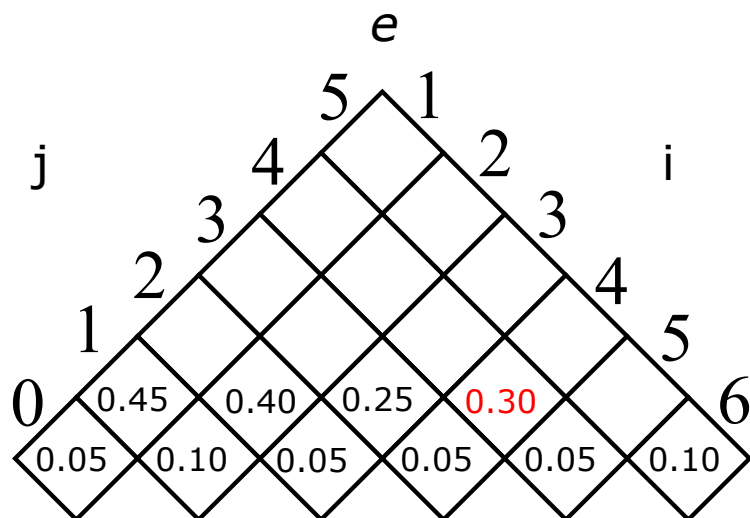
$$e[3, 3] = e[3, 2] + e[4, 3] + w(3, 3)$$

$$e[1, 1] = 0.05 + 0.05 + 0.15$$

$$e[1, 1] = 0.25$$



Optimal binary search trees



i	0	1	2	3	4	5
p_i		0.15	0.10	0.05	0.10	0.20
q_i	0.05	0.10	0.05	0.05	0.05	0.10

$$e[i, j] = e[i, r - 1] + e[r + 1, j] + w(i, j)$$

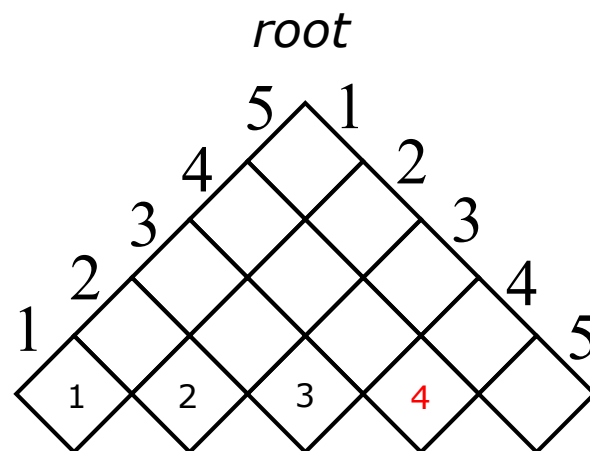
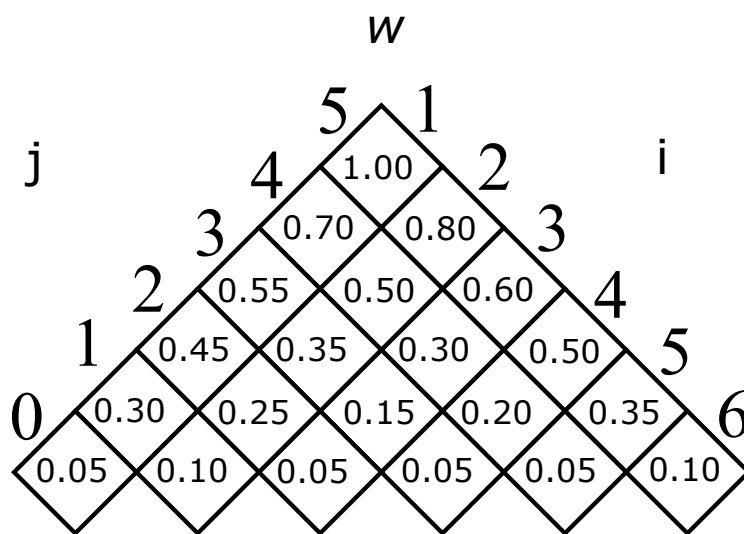
$$i \leq r \leq j$$

$r=4$

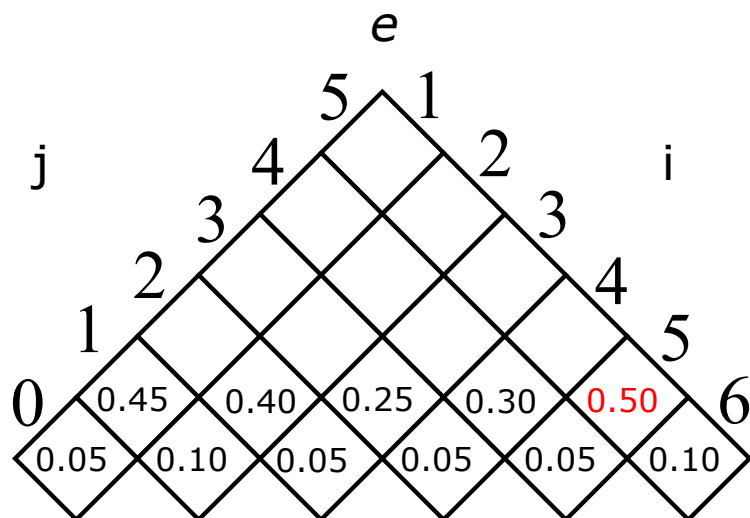
$$e[4, 4] = e[3, 2] + e[4, 3] + w(4, 4)$$

$$e[4, 4] = 0.05 + 0.05 + 0.20$$

$$e[4, 4] = 0.30$$



Optimal binary search trees



i	0	1	2	3	4	5
p_i		0.15	0.10	0.05	0.10	0.20
q_i	0.05	0.10	0.05	0.05	0.05	0.10

$$e[i, j] = e[i, r - 1] + e[r + 1, j] + w(i, j)$$

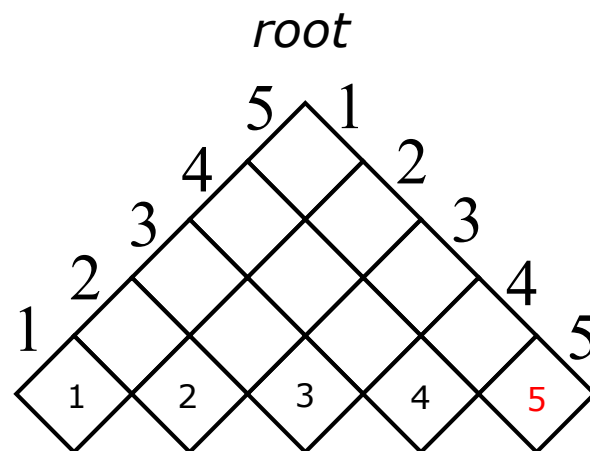
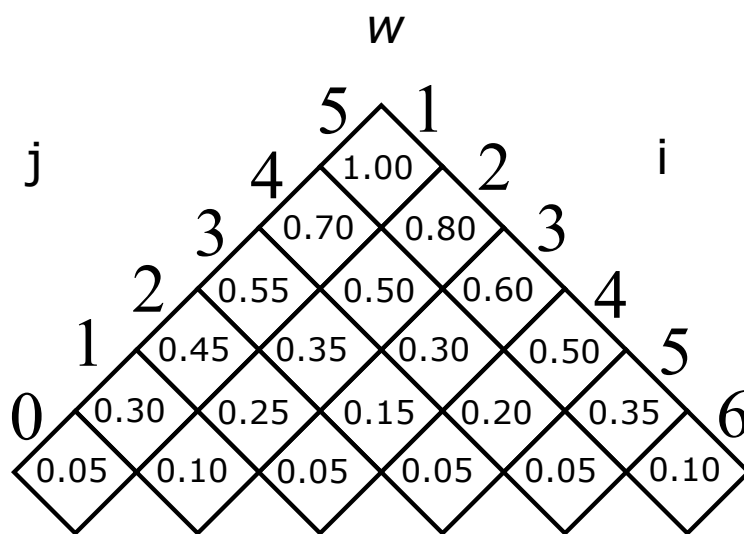
$$i \leq r \leq j$$

$$r=5$$

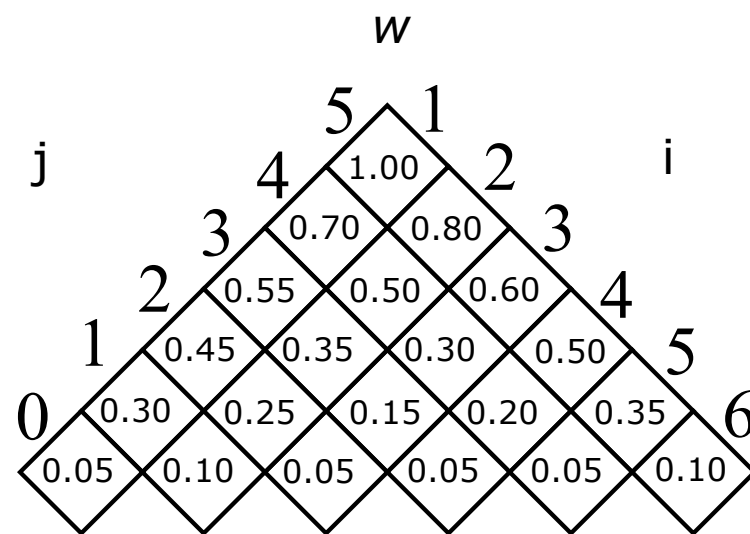
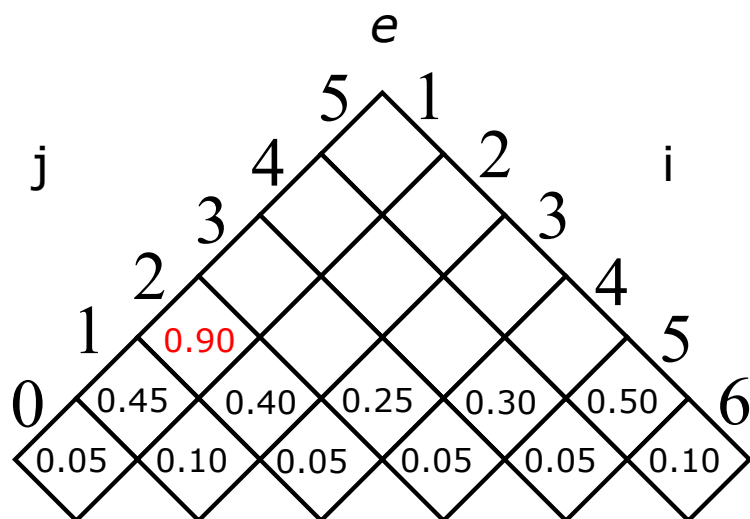
$$e[5, 5] = e[5, 4] + e[6, 5] + w(5, 5)$$

$$e[5, 5] = 0.05 + 0.10 + 0.35$$

$$e[5, 5] = 0.50$$



Optimal binary search trees



i	0	1	2	3	4	5
p_i		0.15	0.10	0.05	0.10	0.20
q_i	0.05	0.10	0.05	0.05	0.05	0.10

$$e[i, j] = e[i, r - 1] + e[r + 1, j] + w(i, j)$$

$$i \leq r \leq j$$

$$r=1$$

$$e[1,2] = e[1,0] + e[2,2] + w(1,2)$$

$$e[1,2] = 0.05 + 0.40 + 0.45$$

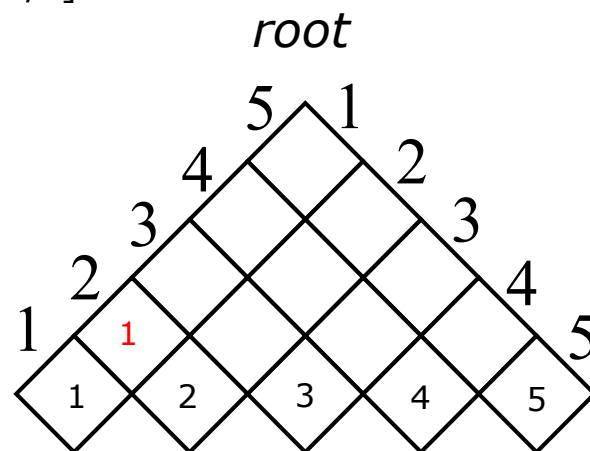
$$e[1,2] = 0.90$$

$$r=2$$

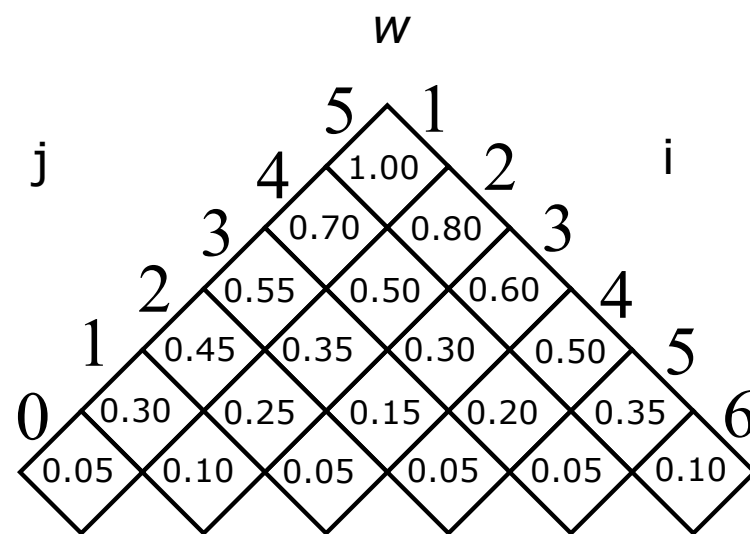
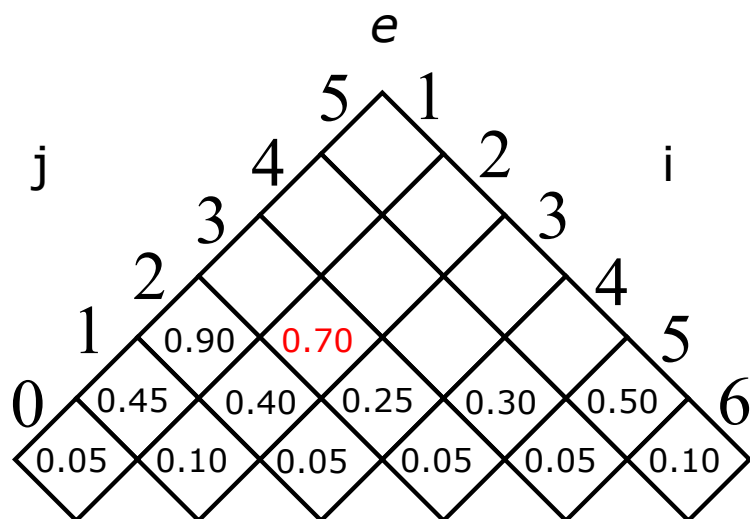
$$e[1,2] = e[1,1] + e[3,2] + w(1,2)$$

$$e[1,2] = 0.45 + 0.05 + 0.45$$

$$e[1,2] = 0.95$$



Optimal binary search trees



i	0	1	2	3	4	5
p_i		0.15	0.10	0.05	0.10	0.20
q_i	0.05	0.10	0.05	0.05	0.05	0.10

$$e[i, j] = e[i, r - 1] + e[r + 1, j] + w(i, j)$$

$$i \leq r \leq j$$

$$r=2$$

$$e[2,3] = e[2,1] + e[3,3] + w(2,3)$$

$$e[2,3] = 0.10 + 0.25 + 0.35$$

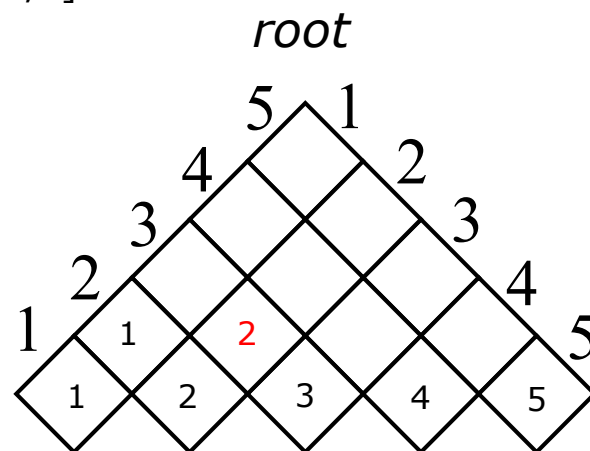
$$e[2,3] = 0.70$$

$$r=3$$

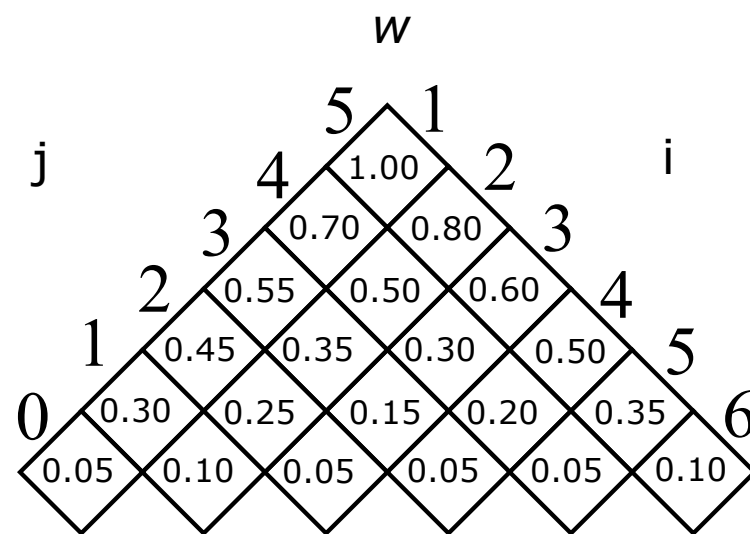
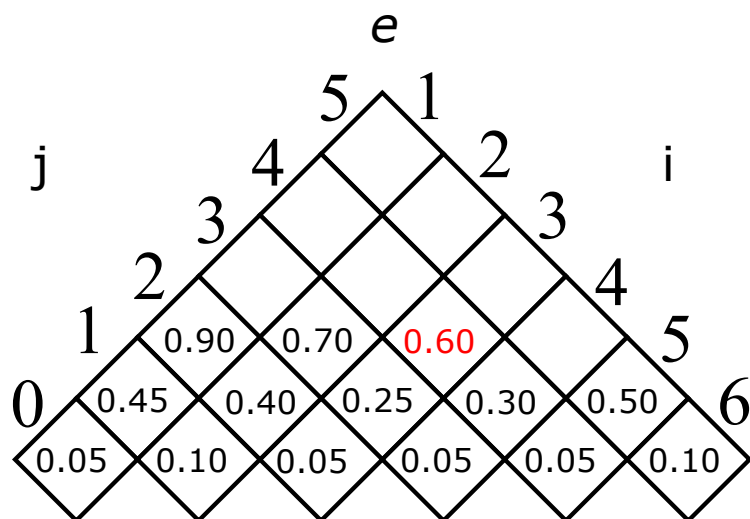
$$e[2,3] = e[2,2] + e[4,3] + w(2,3)$$

$$e[2,3] = 0.40 + 0.05 + 0.35$$

$$e[2,3] = 0.80$$



Optimal binary search trees



i	0	1	2	3	4	5
p_i		0.15	0.10	0.05	0.10	0.20
q_i	0.05	0.10	0.05	0.05	0.05	0.10

$$e[i, j] = e[i, r - 1] + e[r + 1, j] + w(i, j)$$

$$i \leq r \leq j$$

$$r=3$$

$$e[3, 4] = e[3, 2] + e[4, 4] + w(3, 4)$$

$$e[3, 4] = 0.05 + 0.30 + 0.30$$

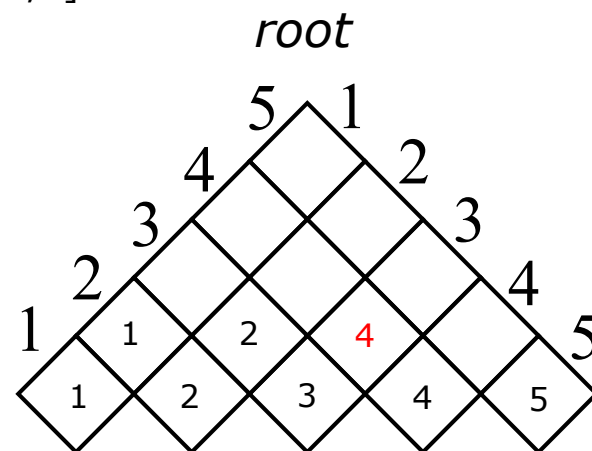
$$e[3, 4] = 0.65$$

$$r=4$$

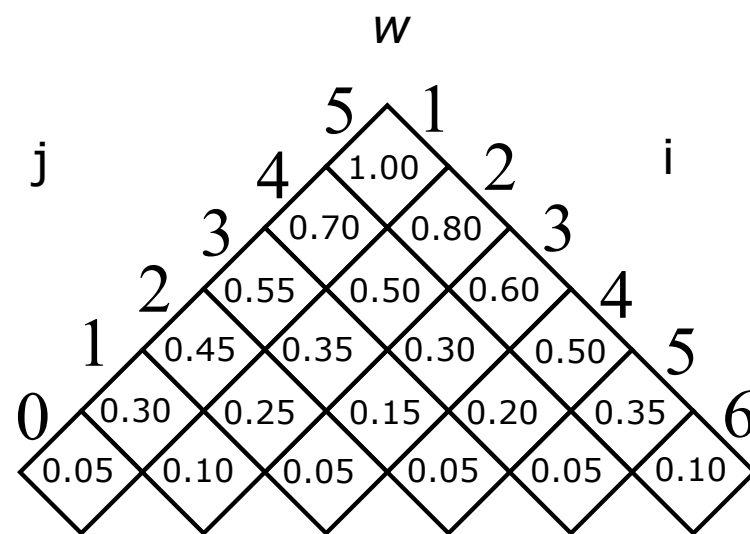
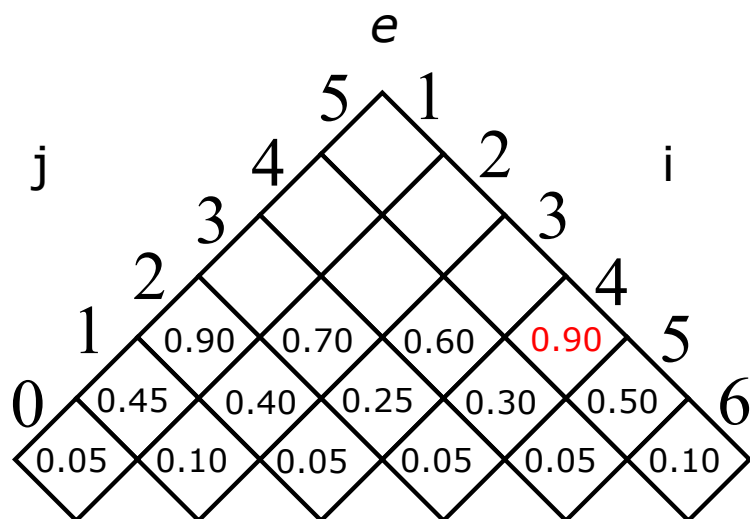
$$e[3, 4] = e[3, 3] + e[5, 4] + w(3, 4)$$

$$e[3, 4] = 0.25 + 0.05 + 0.30$$

$$e[3, 4] = 0.60$$



Optimal binary search trees



i	0	1	2	3	4	5
p_i		0.15	0.10	0.05	0.10	0.20
q_i	0.05	0.10	0.05	0.05	0.05	0.10

$$e[i, j] = e[i, r - 1] + e[r + 1, j] + w(i, j)$$

$$i \leq r \leq j$$

$$r=4$$

$$e[4, 5] = e[4, 3] + e[5, 5] + w(4, 5)$$

$$e[4, 5] = 0.05 + 0.50 + 0.50$$

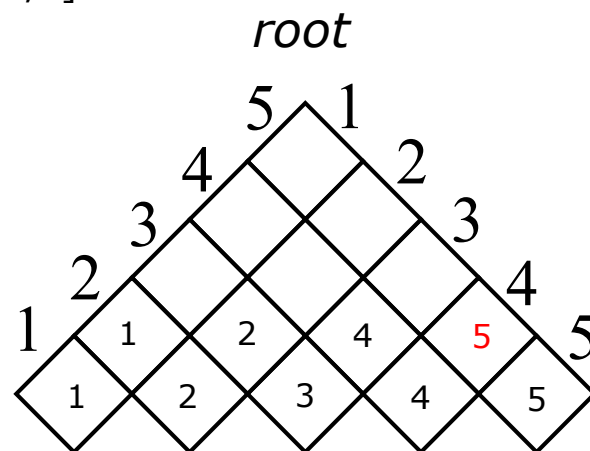
$$e[4, 5] = 1.05$$

$$r=5$$

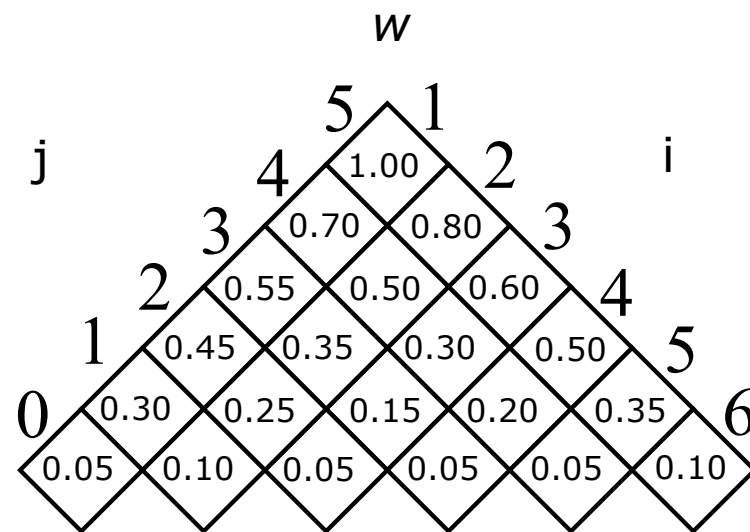
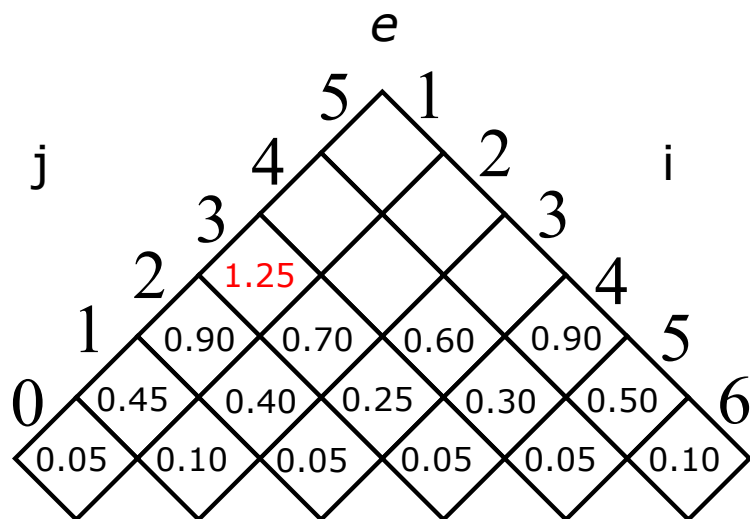
$$e[4, 5] = e[4, 4] + e[6, 5] + w(4, 5)$$

$$e[4, 5] = 0.30 + 0.10 + 0.50$$

$$e[4, 5] = 0.90$$



Optimal binary search trees



$$e[i, j] = e[i, r - 1] + e[r + 1, j] + w(i, j)$$

$$i \leq r \leq j$$

$$r=1$$

$$e[1,3] = e[1,0] + e[2,3] + w(1,3)$$

$$e[1,3] = 0.05 + 0.70 + 0.55$$

$$e[1,3] = 1.30$$

$$r=2$$

$$e[1,3] = e[1,1] + e[3,3] + w(1,3)$$

$$e[1,3] = 0.45 + 0.25 + 0.55$$

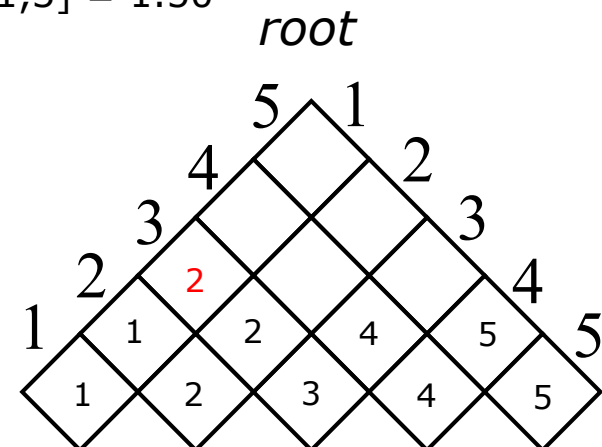
$$e[1,3] = 1.25$$

$$r=3$$

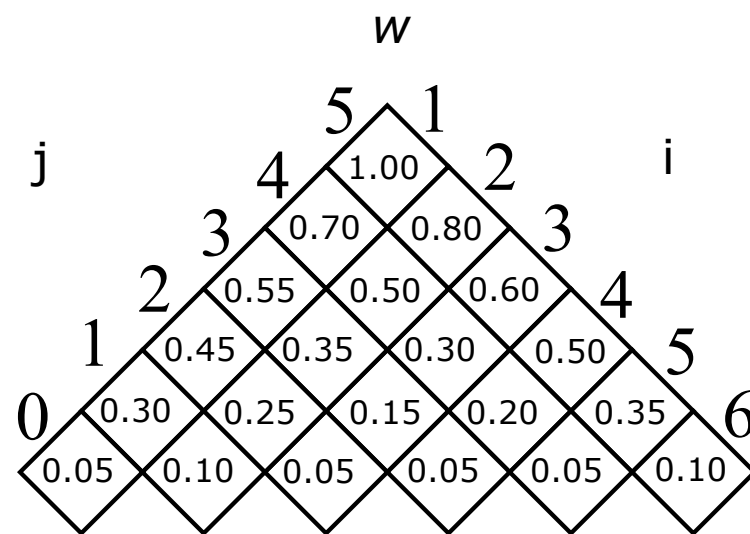
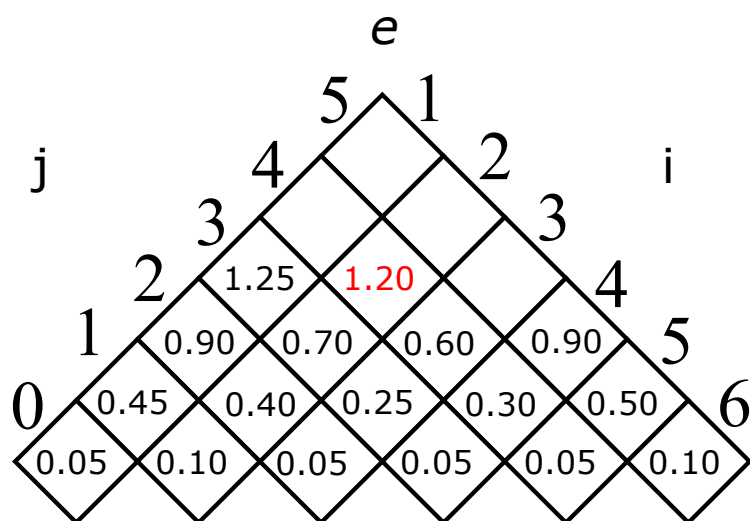
$$e[1,3] = e[1,2] + e[4,3] + w(1,3)$$

$$e[1,3] = 0.90 + 0.05 + 0.55$$

$$e[1,3] = 1.50$$



Optimal binary search trees



$$e[i, j] = e[i, r - 1] + e[r + 1, j] + w(i, j)$$

$$i \leq r \leq j$$

$$r=2$$

$$e[2,4] = e[2,1] + e[3,4] + w(2,4)$$

$$e[2,4] = 0.10 + 0.60 + 0.50$$

$$e[2,4] = 1.20$$

$$r=3$$

$$e[2,4] = e[2,2] + e[4,4] + w(2,4)$$

$$e[2,4] = 0.40 + 0.30 + 0.50$$

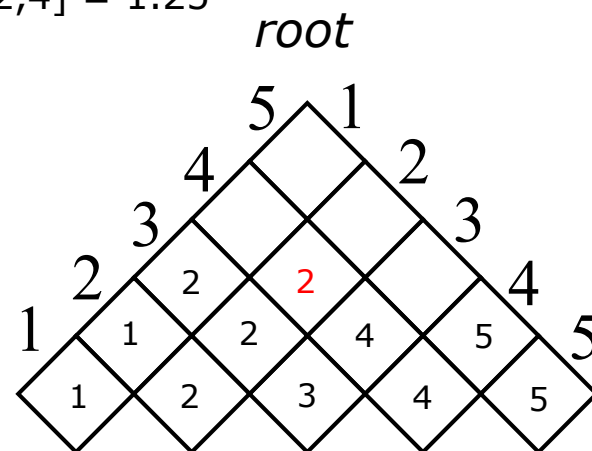
$$e[2,4] = 1.20$$

$$r=4$$

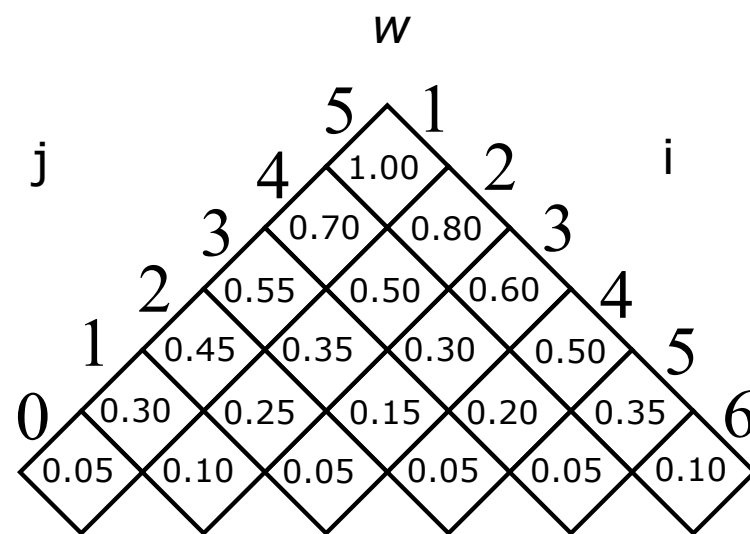
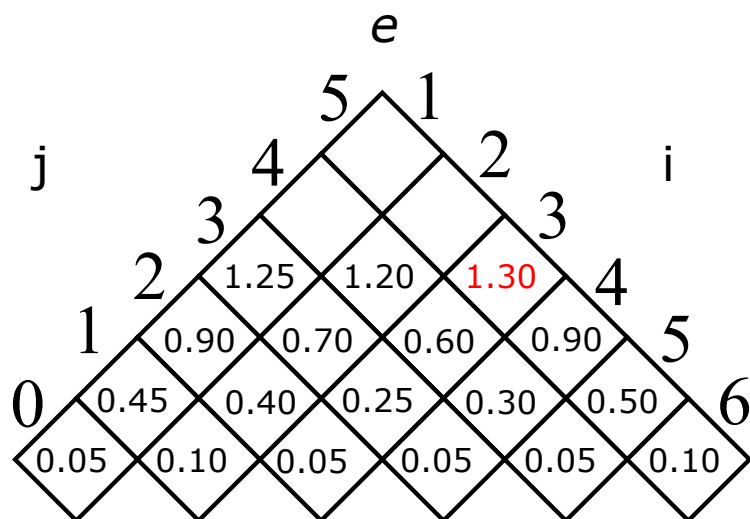
$$e[2,4] = e[2,3] + e[5,4] + w(2,4)$$

$$e[2,4] = 0.70 + 0.05 + 0.50$$

$$e[2,4] = 1.25$$



Optimal binary search trees



$$e[i, j] = e[i, r - 1] + e[r + 1, j] + w(i, j)$$

$$i \leq r \leq j$$

$$r=3$$

$$e[3,5] = e[3,2] + e[4,5] + w(3,5)$$

$$e[3,5] = 0.05 + 0.90 + 0.60$$

$$e[3,5] = 1.55$$

$$r=4$$

$$e[3,5] = e[3,3] + e[5,5] + w(3,5)$$

$$e[3,5] = 0.25 + 0.50 + 0.60$$

$$e[3,5] = 1.35$$

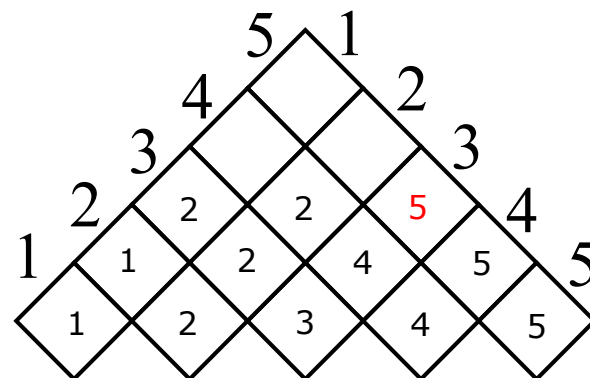
$$r=5$$

$$e[3,5] = e[3,4] + e[6,5] + w(3,5)$$

$$e[3,5] = 0.60 + 0.10 + 0.60$$

$$e[3,5] = 1.30$$

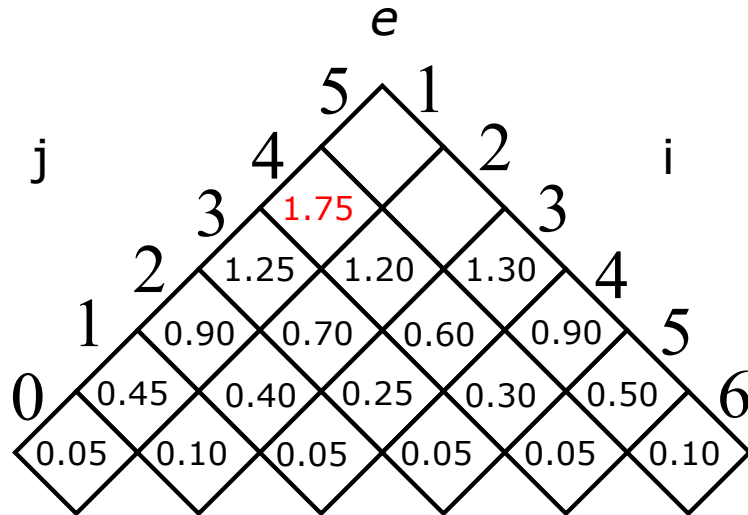
root



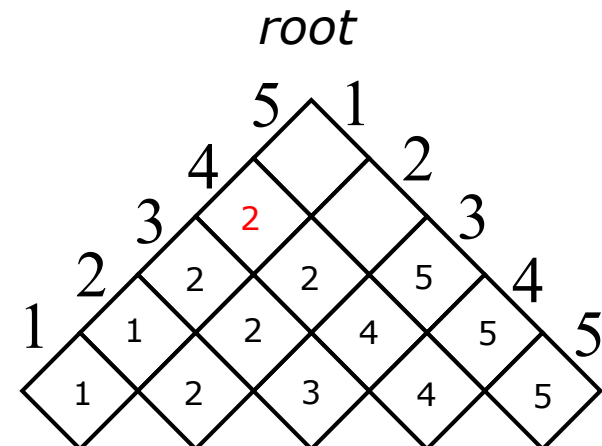
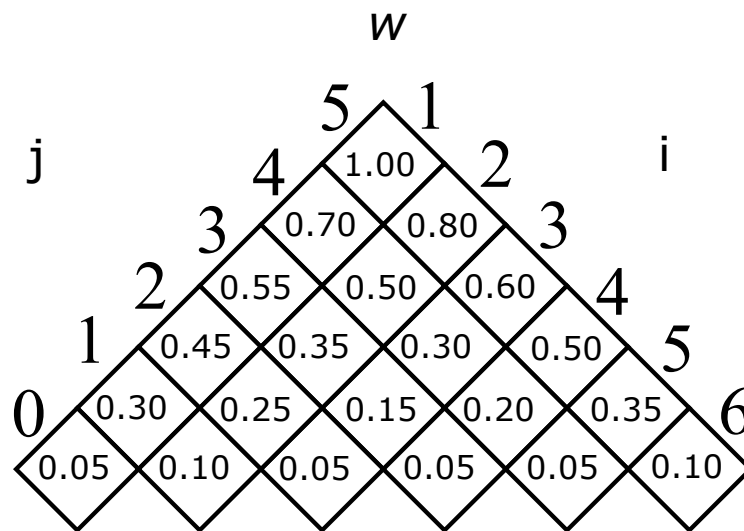
$$e[i, j] = e[i, r - 1] + e[r + 1, j] + w(i, j)$$

$$i \leq r \leq j$$

Optimal binary search trees



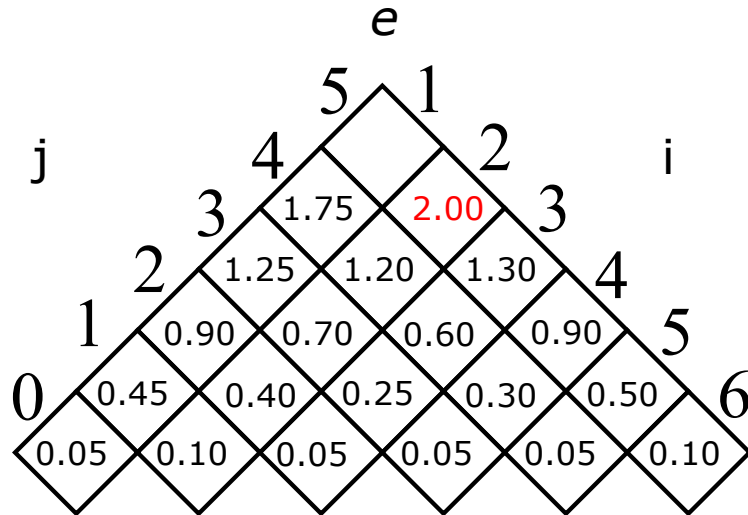
$r=1$
 $e[1, 4] = e[1, 0] + e[2, 4] + w(1, 4)$
 $e[1, 4] = 0.05 + 1.20 + 0.70$
 $e[1, 4] = 1.95$
 $r=2$
 $e[1, 4] = e[1, 1] + e[3, 4] + w(1, 4)$
 $e[1, 4] = 0.45 + 0.60 + 0.70$
 $e[1, 4] = 1.75$
 $r=3$
 $e[1, 4] = e[1, 2] + e[4, 4] + w(1, 4)$
 $e[1, 4] = 0.90 + 0.30 + 0.70$
 $e[1, 4] = 1.90$
 $r=4$
 $e[1, 4] = e[1, 3] + e[5, 4] + w(1, 4)$
 $e[1, 4] = 1.25 + 0.05 + 0.70$
 $e[1, 4] = 2.00$



$$e[i, j] = e[i, r - 1] + e[r + 1, j] + w(i, j)$$

$$i \leq r \leq j$$

Optimal binary search trees

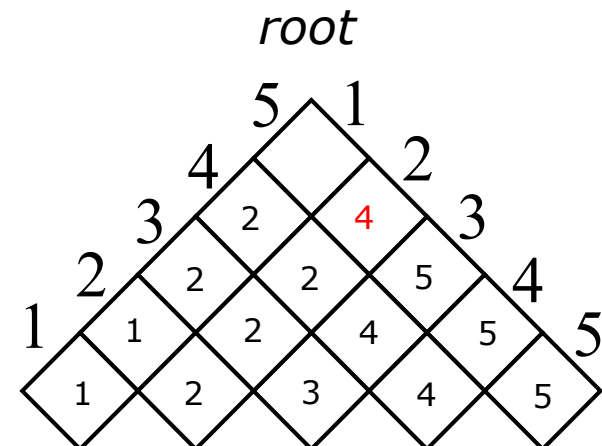
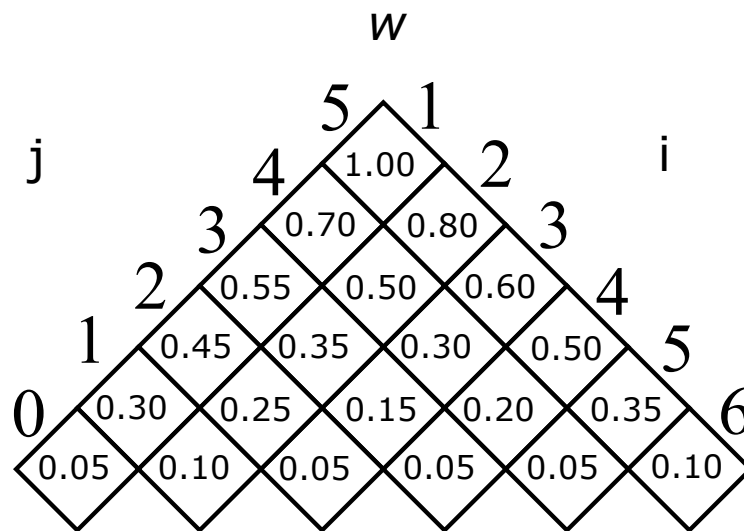


$r=2$
 $e[2,5] = e[2,1] + e[3,5] + w(2,5)$
 $e[2,5] = 0.10 + 1.30 + 0.80$
 $e[2,5] = 2.20$

$r=3$
 $e[2,5] = e[2,2] + e[4,5] + w(2,5)$
 $e[2,5] = 0.40 + 0.90 + 0.80$
 $e[2,5] = 2.10$

$r=4$
 $e[2,5] = e[2,3] + e[5,5] + w(2,5)$
 $e[2,5] = 0.70 + 0.50 + 0.80$
 $e[2,5] = 2.00$

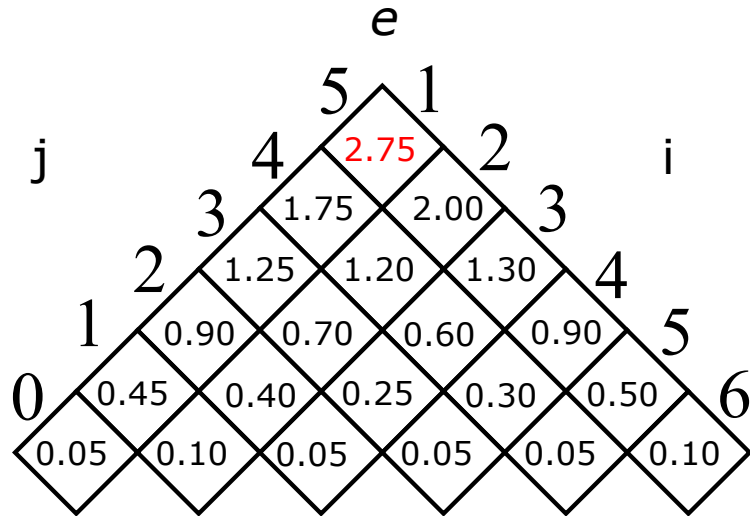
$r=5$
 $e[2,5] = e[2,4] + e[6,5] + w(2,5)$
 $e[2,5] = 1.20 + 0.10 + 0.80$
 $e[2,5] = 2.10$



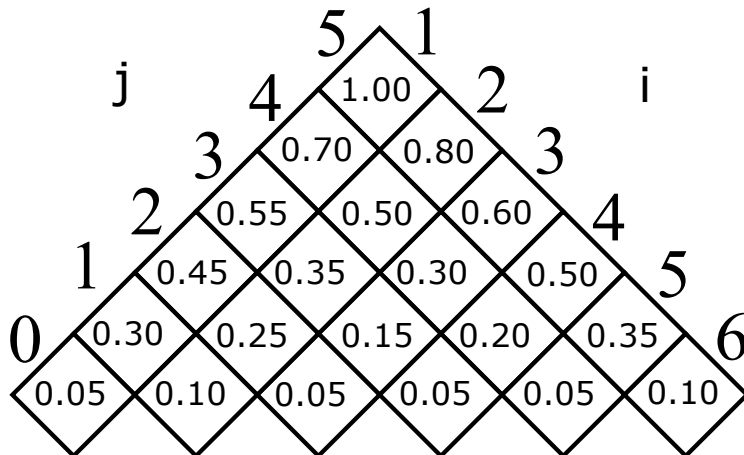
$$e[i, j] = e[i, r - 1] + e[r + 1, j] + w(i, j)$$

$$i \leq r \leq j$$

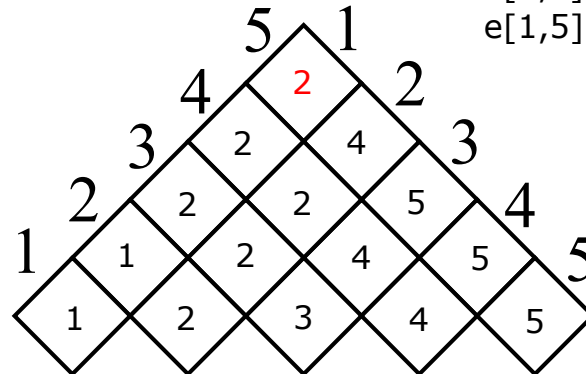
Optimal binary search trees



w



root



$r=1$

$$e[1,5] = e[1,0] + e[2,5] + w(1,5)$$

$$e[1,5] = 0.05 + 2.00 + 1.00$$

$$e[1,5] = 3.05$$

$r=2$

$$e[1,5] = e[1,1] + e[3,5] + w(1,5)$$

$$e[1,5] = 0.45 + 1.30 + 1.00$$

$$e[1,5] = 2.75$$

$r=3$

$$e[1,5] = e[1,2] + e[4,5] + w(1,5)$$

$$e[1,5] = 0.90 + 0.90 + 1.00$$

$$e[1,5] = 2.80$$

$r=4$

$$e[1,5] = e[1,3] + e[5,5] + w(1,5)$$

$$e[1,5] = 1.25 + 0.50 + 1.00$$

$$e[1,5] = 2.75$$

$r=5$

$$e[1,5] = e[1,4] + e[6,5] + w(1,5)$$

$$e[1,5] = 1.75 + 0.10 + 1.00$$

$$e[1,5] = 2.85$$

Optimal binary search trees

□ Step 4: Construct an optimal binary search tree

Given the table $root[i, j]$, the following procedure outputs the structure of an optimal binary search tree.

CONSTRUCT-OPTIMAL-BST($root, n$)

$r = root[1, n]$

print " k " _{r} " is the root"

CONSTRUCT-OPTIMAL-SUBTREE($root, 1, r - 1, r, \text{"left"}$)

CONSTRUCT-OPTIMAL-SUBTREE($root, r + 1, n, r, \text{"right"}$)

CONSTRUCT-OPTIMAL-SUBTREE($root, i, j, r, dir$)

if $j < i$

print " d " _{j} " is the " dir " child of k " _{r}

else $t = root[i, j]$

print " k " _{t} " is the " dir " child of k " _{r}

CONSTRUCT-OPTIMAL-SUBTREE($root, i, t - 1, t, \text{"left"}$)

CONSTRUCT-OPTIMAL-SUBTREE($root, t + 1, j, t, \text{"right"}$)

k_2 is the root

k_1 is the left child of k_2

d_0 is the left child of k_1

d_1 is the right child of k_1

k_5 is the right child of k_2

k_4 is the left child of k_5

k_3 is the left child of k_4

d_2 is the left child of k_3

d_3 is the right child of k_3

d_4 is the right child of k_4

d_5 is the right child of k_5

Optimal binary search trees

CONSTRUCT-OPTIMAL-BST(*root*, *n*)

```

r = root[1, n]
print "k"r " is the root"
CONSTRUCT-OPTIMAL-SUBTREE(root, 1, r - 1, r, "left")
CONSTRUCT-OPTIMAL-SUBTREE(root, r + 1, n, r, "right")

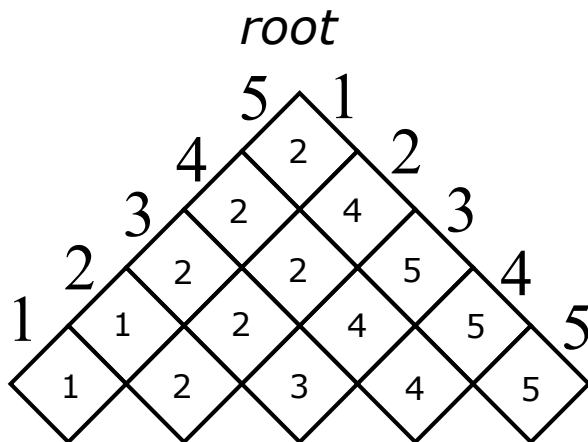
```

CONSTRUCT-OPTIMAL-SUBTREE(*root*, *i*, *j*, *r*, *dir*)

```

if j < i
    print "d"j " is the " dir " child of k"r
else t = root[i, j]
    print "k"t " is the " dir " child of k"r
    CONSTRUCT-OPTIMAL-SUBTREE(root, i, t - 1, t, "left")
    CONSTRUCT-OPTIMAL-SUBTREE(root, t + 1, j, t, "right")

```



r=[1,5]=2

"*k*₂ is the root"

COS(*root*,1,1,2,"left")

t=1 (*root*[1,1])

"*k*₁ is the left child of *k*₂"

COS(*root*,1,0,1,"left")

"*d*₀ is the left child of *k*₁"

COS(*root*,2,1,1"right")

"*d*₁ is the right child of *k*₁"

COS(*root*,3,5,2,"right")

t = 5 (*root*[3,5])

"*K*₅ is the right child of *k*₂"

COS(*root*,3,4,5,"left")

t=4 (*root*[3,4])

"*k*₄ is the left child of *k*₅"

COS(*root*,3,3,4,"left")

t=3 (*root*[3,3])

"*k*₃ is the left child of *k*₄"

COS(*root*, 3,2,3,"left")

"*d*₂ is the left child of

*k*₃"

COS(*root*, 4,3,3,"right")

"*d*₃ is the right child of

*k*₃"

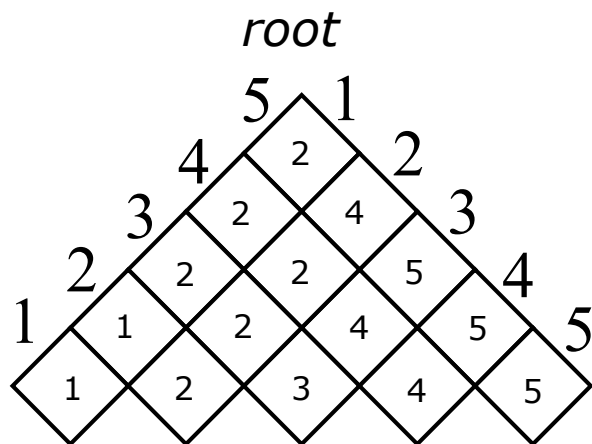
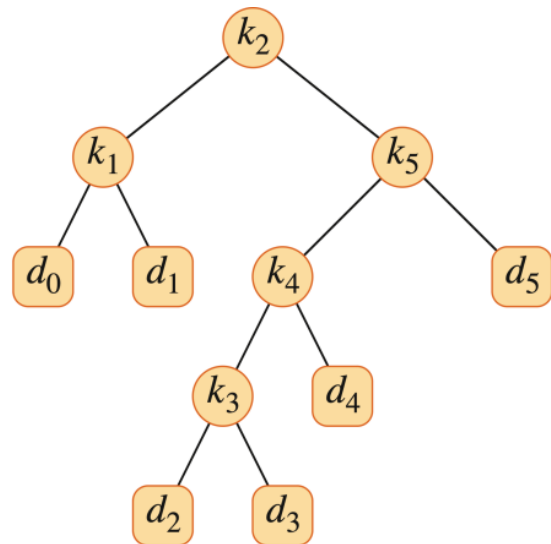
COS(*root*,5,4,4,"right")

"*d*₄ is the right child of *k*₄"

COS(*root*,6,5,5,"right")

"*d*₅ is the right child of *k*₅"

Optimal binary search trees



$r=[1,5]=2$

" k_2 is the root"

$\text{COS}(\text{root}, 1, 1, 2, \text{"left"})$

$t=1$ ($\text{root}[1,1]$)

" k_1 is the left child of k_2 "

$\text{COS}(\text{root}, 1, 0, 1, \text{"left"})$

" d_0 is the left child of k_1 "

$\text{COS}(\text{root}, 2, 1, 1, \text{"right"})$

" d_1 is the right child of k_1 "

$\text{COS}(\text{root}, 3, 5, 2, \text{"right"})$

$t=5$ ($\text{root}[3,5]$)

" k_5 is the right child of k_2 "

$\text{COS}(\text{root}, 3, 4, 5, \text{"left"})$

$t=4$ ($\text{root}[3,4]$)

" k_4 is the left child of k_5 "

$\text{COS}(\text{root}, 3, 3, 4, \text{"left"})$

$t=3$ ($\text{root}[3,3]$)

" k_3 is the left child of k_4 "

$\text{COS}(\text{root}, 3, 2, 3, \text{"left"})$

" d_2 is the left child of

k_3 "

$\text{COS}(\text{root}, 4, 3, 3, \text{"right"})$

" d_3 is the right child of

k_3 "

$\text{COS}(\text{root}, 5, 4, 4, \text{"right"})$

" d_4 is the right child of k_4 "

$\text{COS}(\text{root}, 6, 5, 5, \text{"right"})$

" d_5 is the right child of k_5 "

Reading

□ Section 14.5.